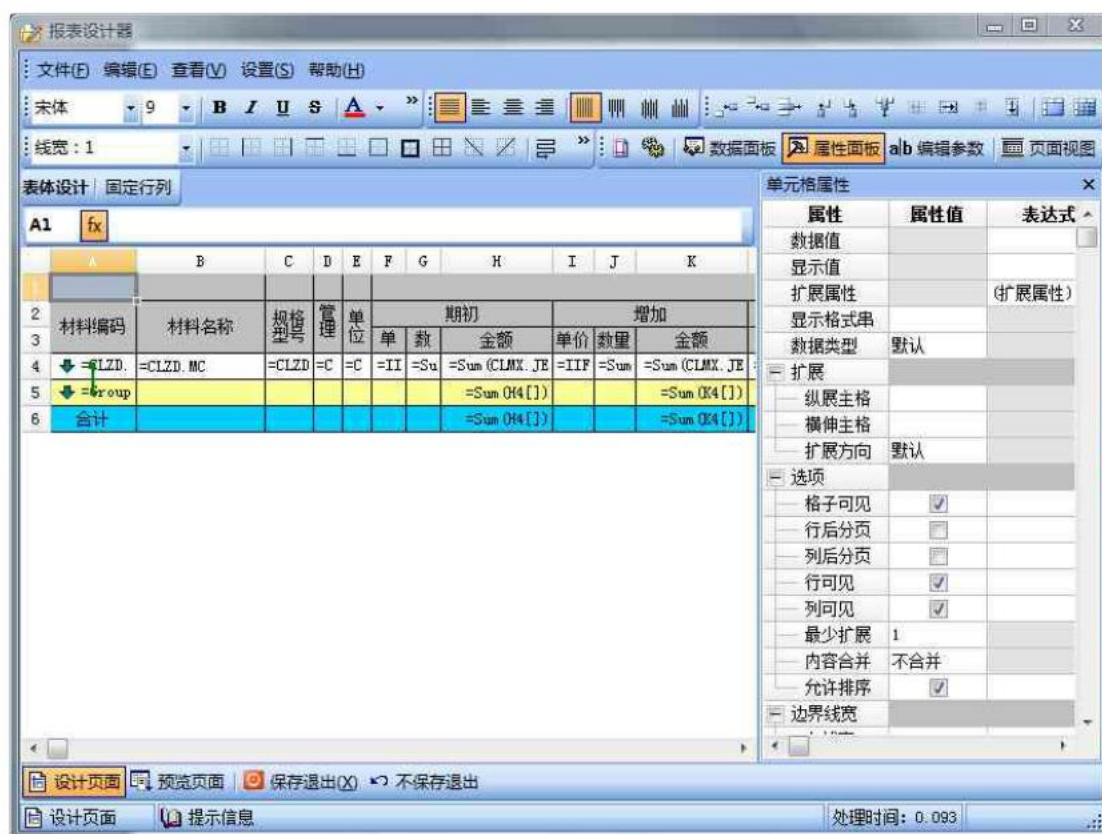




报表引擎设计

Glodon 广联达 周金根

zhoujingen@gmail.com

<http://zhoujg.cnblogs.com/>

2010-2-25

目录

报表引擎设计.....	1
前言	4
报表应用思路.....	5
提出问题.....	5
前期.....	5
报表引擎带来的好处.....	6
以前不能满足的部分功能.....	6
建议.....	6
存在风险.....	7
主要思路.....	7
范围.....	7
争对用户.....	7
报表类型.....	7
报表制作方式.....	8
主要技术.....	8
设计说明（主要从一些概念来说明）	13
基本概念.....	13
可扩展报表.....	13
填报表.....	24
报表模型概述.....	25
介绍.....	26
总体组件图.....	26
主要产品组成.....	26
报表设计器.....	26
报表服务器.....	27
报表显示控件.....	27
架构描述.....	29
介绍.....	29
架构目标和约束.....	29
可维护性和可扩展性.....	29
可用性.....	29
易用性.....	29
实现约束.....	29
架构表现.....	29
用例模型.....	30
分析模型.....	31
设计模型.....	32
扩展模型.....	34
介绍.....	34
术语.....	34
扩展概念.....	34
可扩展单元格.....	34

横向扩展.....	34
纵向扩展.....	36
不可扩展.....	38
默认.....	38
横伸主格.....	38
纵展主格.....	39
主单元格和附属单元格.....	41
规则.....	41
单元格的层次坐标和汇总运算.....	42
绝对层次坐标.....	42
相对层次坐标（位移坐标）.....	45
条件表达式.....	46
& 运算符.....	47
? 运算符.....	48
根坐标表示法.....	49
命名规则.....	49
其他规则.....	49
数据模型.....	50
介绍.....	50
术语.....	50
操作符.....	50
集合函数.....	52
其他.....	52
表格运算示例.....	52
约束.....	52
页面宏.....	52
页统计.....	52
算法说明.....	53
单元格变量.....	53
单元格列表.....	53
图形模型.....	54
介绍.....	54
术语.....	54
矢量图.....	54
图片.....	54
页面背景图.....	54
单元格背景图.....	55
单元格图片内容.....	55
统计图.....	55
设置.....	55
打印模型.....	57
介绍.....	57
术语.....	57
自动折行.....	60

扩展.....	60
打印规则.....	60
交叉表的设计示例.....	62
附一：研究润乾报表的实现.....	63
附二：开源 — 轻型的表达式引擎 Flee.....	64
优点.....	64
如何使用.....	64
Flee 介绍.....	64
附三：分析能力的 8 个等级.....	66
附四：图表建议.....	68

前言

报表管理软件中常用的基础组件之一，简单报表可以轻易的通过一些方式实现，但想要处理国内复杂的报表，则必须开发一个报表引擎，本文将从报表引擎的设计方面进行讲解，只要掌握了这些核心模型（借鉴润乾报表思路），就可以实现一个比较好的引擎。

欢迎大家来信 zhoujingen@gmail.com 提出你的建议。

报表应用思路

[OpenExpressApp](#) 计划之内包括一个报表引擎 OpenReport。2006 年时在公司负责研究并实现了一个报表引擎，写了几篇博文（[《交叉表的报表设计》](#)、[《研究润乾报表的实现》](#)、[《使用 Antlr 实现表达式引擎》](#)）。OpenReport 的报表模型将会和我已经实现的差不多，但是有些主要差别就是，OpenReport 除了支持数据集，还可以基于对象，在设计器上和使 用上将会更易用更方便，会更多考虑业务人员的使用和一些 BI。但是实现起来会花更多时间和精力，而现在的精力毕竟有限，所以[还没有开始 OpenReport 的工作，目前 OEA 主要还是集中在应用框架上。](#)

现在还有人看到我以前的两篇和报表相关的博文并非常感兴趣，但由于那个报表引擎不是开源产品，所以不能公布代码，在这里[我就将以前写的一些文档发布一下](#)，给那些需要或者感兴趣的人借鉴一 下。我之前也是看了别人的一些文档而作出来的这个报表引擎，所以研究报表引擎的看完这些文档之后应该就会有大体感觉。

这个系列将会发布多篇，感兴趣的可以留意一下。本篇为第一篇，将从总体上介绍一下报表应用思路，这是我 2006 年 1 月份在刚开始研究报表几周后写的。就像很早以前博文[《研究润乾报表的实现》](#)所说，这里的模型主要是参考润乾报表的，我觉得它是我研究时期看到的最好的模型。现在润乾有一个免费版本[快速报表](#)，强烈建议研究报表的去用一下，对项目有需求的也可以直接用，在这里也希望润乾报表能做的更好更易用。

提出问题

报表对于任何管理软件来说都是必不可少的一项主要功能，对于成本管理系统来说企业对报表的要求也逐渐加强，现有公司的报表控件已经在功能和开发效率上不能 完全满足要求，希望通过分析能够提出改善的报表解决方案，在能够满足客户需求的情况下再尽量减少开发人员的工作量，并提高软件的质量。

前期

通过两周时间，收集了一些软件公司的报表软件，分析了一下水晶报表、SAP 程序报表以及国内的一些报表厂商的软件，如润乾软件、博易智软等。分析后觉得润 乾报表的思路比较好，后期主要针对润乾报表的一些思路进行分析。以下将把主要思路罗列出来，可以作为后期深入的参考。

报表引擎带来的好处

1. 分离报表实际业务数据和展现形式，只需准备源数据，后续数据展现工作由报表引擎来完成。比如一些小计行，字段合并，大小写等都不应该写在 SQL 语句中
2. 采用多源分片和动态格间计算技术能够高效的完成复杂报表工作，缩短产品开发周期，提高产品质量
3. 通过提供的业务对象和简单的 SQL 语句构造向导，可以让业务人员制作简易报表
4. 考虑数据上报功能，系统解决项目和公司的数据来往业务，不需要给每个公司报表另单独做一个模块
5. 原来单据中编码会夹杂报表功能代码，报表引擎可以分离报表和实际单据业务功能，使得模块功能更清晰，可维护性更高
6. 提供报表推拉模式，可以由用户订阅关心的报表，使得能够方便的查看需要的数据
7. 通过提供保存查询条件的功能，可以使得公司负责人员打开报表就能看到预期的结果，免去重复性的输入

以前不能满足的部分功能

以下简单罗列了在以前在 XXX 系统不能很好解决的几个问题，未收集造价产品的需求：

1. 颜色警告控制需要代码编写：在相应格子的颜色属性的表达式设置可以解决此问题
2. 通过查询条件的设置来设定数据显示列的是否显示：通过交互报表的设计，在相关列上的 Visible 属性上输入与选择条件相关的表达式
3. 动态列：报表引擎提供横向扩展和纵向扩展，解决动态列将是非常简单的一件事情
4. 需要小计、合计功能：采用动态格间计算，对于小计和合计等功能非常简单的就能设计好
5. 主细表的报表需要编写代码完成：通过报表引擎的交互报表解决或者使用树型支持变通报表样式解决
6. 不能对展现后的数据进行过滤，组合操作：在报表控件中添加对本地报表缓存数据的处理，提供过滤组合功能
7. 数字大小写：通过提供字符串和数字表达式等在格子显示值中进行转换轻易解决
8. 对于公司级要求的报表都是定制开发单个模块：如果考虑上报功能，可以统一解决此类问题

建议

1. 在 VSS 的专题研究\报表\行业软件\润乾软件\runqian_report 下有一个《单位领料明细汇总查询》的示例，以前做此单据不算维护工作量也需要三四天的时间，如果按照此思路完成报表工具，对于熟悉产品的开发人员在半天就能准确的完成此报表，而且可维护性高。虽然未考虑全面，但从目前来看，此思路在整体上较好，方案也是可行的。

2. 润乾报表为 Java 产品，选择条件面板支持不够,还未发现较好的商业报表产品可以购买,建议公司自主开发。
3. 报表功能整体涉及面较大，但每部分在我们业务都是需要的，建议公司安排有报表经验的专人深入分析后分步骤开发，先完成下文[主要技术](#)中说明的几个问题。

存在风险

1. 此文档主要涉及报表数据相关的一些概念部分，对于报表整个系统还未考虑完全。
2. 只参考了几个查询模块，未深入考虑具体情况，比如通过查询条件构造内部 SQL 语句的执行、报表交互的控制和扩展、表达式的解析以及上报系统的深入分析，具体深入后可能会发现其他问题。
3. 开发时间可能会和 XXX4 整体进度冲突，如有冲突需要另考虑 XXX4 报表的过渡方案

主要思路

范围

报表在系统中分为两类：仅供用来查询显示的报表和提供上报公司的报表。目前主要争对查询类分析，对于上报类也将涉及部分。如未显示说明，以下所说的都争对 的是查询类分析。在报表使用方面，主要争对报表设计上，本文档不包含系统对报表的管理，比如报表的权限、报表的目录层次等。

文档将涉及普通报表、交互报表制作以及选择条件的一些思路。

争对用户

主要争对开发人员，提高开发效率和软件质量。在不考虑效率的情况下允许业务人员制作大量简易报表，遇到效率问题和复杂报表时仍由开发人员完成。

1. 开发人员：报表开发完成后，通过报表设计完成目前所有的查询界面。减少大量组合 SQL 语句的编写，提高软件质量。分离表现和业务关注点，使开发人员关注业务数据的获取。
2. 实施人员：经过培训可以完成复杂的报表
3. 业务人员：可以自己完成中等复杂的报表

报表类型

采用多源分片、动态格间计算主要技术，使用类似 Excel 的 Cell 样式进行报表数据的展现。后续会说明这几个技术的概念。

报表制作方式

1. 报表设计工具：大部分的报表工作都在这里进行。系统默认提供网格、交叉、分组样式
2. 编码：对于前期没有考虑不能使用报表工作制作的报表

主要技术

1. 报表引擎
2. 表达式解释
3. 控件开发
4. 脚本语言扩展

以下主要争对报表引擎本身进行说明（多源分片和动态格间计算主要参考的是润乾报表的思路）。

多源分片

结算、支付、合同费用类型、单位等数据都存放在不同表中，传统开发中是在服务器端通过连接或者 Union 来构造复杂的 SQL 语句。这是一种单源不分片的方法，在涉及表多的时候会带来两大问题：

1. SQL 的复杂程度：横向分片，需要 JOIN；如果纵向也有分片，则需要 JOIN+UNION。如果数据来自于五六个数据源，写这个 SQL 就需要水平和时间，也不利于维护
2. 运行效率：涉及多张表时，就算复杂的 SQL 正确的写出来了，但由于 JOIN 和编写 SQL 的写法，实际运行时的效率比较低

通过获取简单的几个表，然后再设计器中通过设定格子的扩展、值表达式和显示表达式，即可完成这张报表。合同单据、单位字典等 SQL 语句可以通过系统的业务对象获取，还起到复用作用，在不破坏报表的情况下，当需求更改时只需要更改业务对象本身即可。

分片是指报表的纵向或横向（或双向）同时被分成了多个区域，每个区域重复规则不同，而又可能相互运算。许多传统报表工具都提供子报表。子报表可以说是某一种分片的方法，但是子报表与主报表并非一个整体，不能互相运算。这里说的分片，是各片处于一个统一的报表当中，可以互相运算。

消耗材料领料出库单

材料类别：主材/地材

领料单位：福田区正达五金商店

出库日期：2005-12-27

单据编号：地材-20051227-001

入库单编号	材料编码	材料名称	规格型号	单位	可用库存	出库数量	出库单价	出库金额	说明
地材-20051227-001	Z0305007	墙砖	200*300	块	100	90	100.00	9000	
合 计：	玖仟圆整								
施工部位：									
备 注：									

编制人：管理员

编制日期：2005-12-27

审核

XXX3 中对于主表汇总细表金额都是在外部计算好后以宏的方式传入报表，不能在报表本身设计。在多源分片的概念下，将可以设置合资格子的表达式，通过细表的明细汇总出金额。

对于数据源的定义，可以通过一个数据库 SQL 语句生成器来生成简单的 SQL 语句。选择条件（过滤语句）可以在 SQL 语句中指定，也可以由报表引擎自动根据条件选择设置下推到数据库。（此问题我还未仔细考虑，只是粗略的想法）

动态格间计算

所有的报表工具都会提供一些计算列的功能，在原始数据基础上再计算出一些别的列值或统计值，比如小计合计等。这是报表展现中是不可缺少的功能。但是，传统工具在计算上的机制是有局限的：

1. 传统工具一般只提供同行内的格间运算和针对某组（或全体）的集合运算，对于常见的跨行组运算则相当困难。
2. 个别传统工具提供了简单的跨行能力，如可以引用上一行数据，而跨组则无能为力。

3. 对于集合运算只提供个别固定的函数，如取第一名、算累计值等，无法组合通用集合运算，如取第二名、算累计的乘积等。

4. 不能实现某些带条件的运算，如计算语文成绩在 90 分以上的同学的数学成绩总和。

5. 除了有规律的跨行组运算外，报表中还可能会有一些随意的独立格运算，其值可能是报表中的任意几个其它格运算出来，甚至还可能会引用到报表外的数据（比如和数据库中的数据再次运算等），这些都很难实现。

这些问题的根本原因在于，传统工具没有很好的运算后报表数据命名机制（传统工具只能用列名命名设计阶段的数据单元），很难精确描述数据引用关系，只能写出规律性很强的表达式，但随意的独立格运算会使传统工具无法处理，而报表外的数据引用更是只能借助脚本或外围应用程序，导致代码极为混乱。而在具有动态格间运算的报表中可以更加灵活地处理很多报表中的运算问题，可以不受上述种种限制，灵活地进行计算。并且，这种计算不仅可以随意地在数据格间进行，还可以在有数据展开的情况下进行动态的引用。

对于以下示例，表尾需要进行统计，如果在以前的报表中处理则一般是先通过程序编码计算累计值，然后通过宏传入报表。

地区	合计		固定电话		小灵通		移动电话	
	销售额	排名	销售额	排名	销售额	排名	销售额	排名
朝阳	972,102,181.00	1	9,879,866.00	2	897,868,567.00	1	84,353,748.00	4
崇文	194,828,511.00	2	8,567,857.00	4	98,886,090.00	2	87,574,564.00	2
东城	12,516,819.00	8	3,452,345.00	8	8,987,578.00	5	96,896.00	8
丰台	72,399,706.00	5	7,646,346.00	5	78,575.00	8	84,674,785.00	3
海淀	101,782,940.00	4	6,345,768.00	6	89,768,585.00	3	5,868,587.00	7
石景山	161,115,195.00	3	9,879,868.00	1	83,547,869.00	4	87,687,458.00	1
西城	13,003,779.00	7	5,668,746.00	7	887,575.00	7	6,467,458.00	6
宣武	22,898,007.00	6	9,876,757.00	3	8,485,896.00	6	6,535,354.00	5
前三名城区销售额总和：1,328,0						占总额比例：85.64%		
城八区占北京市的销售比例：79.49%								

在有格间计算的报表工具中，实现起来将非常简单。在计算排名时，C4格使用动态层次坐标表达式=count(B4[0;0]{B4>\$b4}))+1算出当前排名。在统计部分，简单的使用了统计表达式的数据过滤来完成，如=sum(b4{c4<=3})，计算排名号码在 3（包含）一下的销售额，即总计。设计如下：

C:\Documents and Settings\wq\My Documents\我接收到的文件\城八区电信销售统计表.raq					
	A	B	C	D	E
1	城八区电信销售统计表				
2	地区	合计		= ds1.group(类别, false)	
3		销售额	排名	销售额	排名
4	= ds1.group(	=sum(E4{}}	=count(B4[0:0]{B4	= ds1.sum(销售	=count(E4[0
5					
6(尾)	前三名城区销售额总和：=sum(b4{c4<=3})			占总额比例：=c6/sum(b4{}	
7(尾)	城八区占北京市的销售比例：=sum(b4{})/ ds2.sel				

条件选择窗体

1. 选择参数，允许用户输入单值，程序可以根据该值进行选择
2. 复杂选择标准： 可以包括范围、操作数、选择条件（大于、小于、大于或等于等）
3. 定义输入帮助和搜寻帮助（比如选择材料按 F4 则出现一个下拉材料列表以便选择）
4. 保存选择条件，以供下次运行程序调用或者直接使用存储参数调用报表

选择条件元素格式化

1. 参数格式化：缺省值、是否必须、输入值检查、是否显示等
2. 其他元素：空行、下划线、注释文字
3. 调整元素位置

交互报表功能

系统需要的报表可能不只是用来查看，还需要一些交互功能。比如

1. 报表钻取功能，可以在一个报表调用另一个报表或者系统的单据
2. 报表内部交互，比如切换材料的单位等。或者同一报表有一个材料类别列表，切换后显示相应类别下的材料信息等。
3. 报表查询结果和选择窗体的交互，比如根据报表选择窗体的设置来设定报表的显示样式等

设计说明（主要从一些概念来说明）

基本概念

1. 单元格：报表由行列整齐的格子组成，这些格子我们称为单元格，所有的单元格组成了报表。类似于 Excel 的每个格子
2. 合并单元格：多个连续的单元格通过合并操作后显示为一个大的格子，合并后的格子成为合并单元格
3. 单元格数据类型：支持普通文字、统计图、图片字段、图片文件、子报表、空白单元格、HTML 文本
4. 单元格属性的属性值和表达式：单元格的每个属性都有属性值和表达式两种（属性值和表达式之间互斥，定义了属性值，就不必定义表达式，定义了表达式，就不

必定义属性值)。属性值是在编辑报表时就写死了的，在报表运行时不必经过运算，也不会被改变，而属性的表达式在报表运行时，会被运算，该属性的最终属性值

取决于表达式的运算结果值。在属性表达式中可以用"@value"来引用单元格的数据值。

5. 数据集：可以是一个 SQL 语句定义的二维表，也可以通过业务对象获取（比如材料字典、入库单等）
6. 参数：获取数据前用来过滤报表数据的元素，实际上作用在 SQL 语句的 where 或参数中。参数有数据类型、可以有缺省值和其他格式设定等。参数可以在数据集和表达式中被引用。
7. 宏：宏代表没有数据类型的一个字符串，在报表运算时将用宏值替换宏变量所占的位置。可以在报表中任何位置引用宏变量，如单元格值、表达式、数据集定义、单元格属性表达式中等，引用方法为\${宏变量名}

可扩展报表

横向扩展

单元格的表达式返回多个数据值(比如合作单位集合列表)时，该单元格可以横向进行复制，复制单元格的数据值依次为表达式的结果数据值，表达式返回几个值，单元格就复制几个，这种现象称为单元格的横向扩展。

横向扩展时，下方单元格进行复制，复制时，复制出来的新单元格的所有属性都引用被复制单元格的属性；纵向扩展时，上方单元格的宽度进行延展，延展成复制后的所有单元格的宽度总和；纵向扩展时，下方的单元格如果从属于高于当前单元格级别的单元格，则该下方单元格也应该延展，而非复制

例 1：横向扩展

扩展前：

	B1	C1
	B2	C2
	B3	C3

扩展后：

	B1			F1
	B2			F2
	B3			F3

例 2：下方较高级别单元格延展

假设 B2 是 B1 的附属单元格，B4 也是 B1 的附属单元格，因此从逻辑上讲，B4 至少和 B2 平级，当 B2 进行扩展时，B4 应该延展，不应该复制

扩展前：

	B1	C1
	B2	C2
	B3	C3
	B4	C4

扩展后：

	B1			F1
	B2			F2
	B3			F3
	B4			

纵向扩展

单元格的表达式返回多个数据值时，该单元格可以纵向进行复制，复制单元格的数据值依次为表达式的结果数据值，表达式返回几个值，单元格就复制几个，这种现象称为单元格的纵向扩展。纵向扩展时，右边单元格进行复制，复制时，复制出来的新单元格的所有属性都引用被复制单元格的属性；纵向扩展

时，左边单元格的高 度进行延展，延展成复制后的所有单元格的高度总和； 纵向扩展时，右边的单元格如果从属于高于当前单元格级别的单元格，则该右边单元格也应该延展，而非复制。

例 1：纵向扩展

扩展前：

A1	B1	C1
A2	B2	C2
A3	B3	C3

扩展后：

A1	B1	C1
A2	B2	C2
A5	B5	C5

例 2：右边较高级别单元格延展

扩展前：

A1	B1	C1	D1
A2	B2	C2	D2
A3	B3	C3	D3

扩展后：

A1	B1	C1	D1
A2	B2	C2	D2

A5	B5	C5	D5

左主格

单元格纵向扩展时，有时候需要跟着被复制的单元格不一定在该单元格的右边，此时，需要对被复制的单元格设置左主格属性。

A 单元格纵向扩展时，B 单元格被同步复制，此时 A 单元格称为 B 单元格的左主格。B 单元格称为 A 单元格的附属单元格。左主格扩展时，附属单元格被纵向同步复制，且与左主格的相对位置保持不变，附属单元格本身的附属单元格也将被复制。复制时，复制出来的新单元格的所有属性都引用被复制单元格的属性；在实际的报表逻辑中，附属单元格往往是主单元格在同一维上的更细划分，或者是主单元格当前维度的聚集，有时候附属单元格可能是主单元格的标题。

例 1：附属单元格是主单元格的更细划分

假设：A2 是 A3 的左主格，A3 是 A4 的左主格

A1	B1
A2（年）	B2
A3（季度）	B3
A4（月份）	B4

扩展后：

A1	B1
2000 年	
第一季	
1 月	
2 月	
3 月	
第二季	
4 月	
5 月	
6 月	
第三季	
7 月	

例 2：附属单元格是主单元格的在当前维度的聚集

假设 A3 是 A2 单元格的附属单元格，A2 是 A3 的左主格

A1	B1	C1
A2（季度）	B2（月份）	C2
A3（小计）		C3

扩展后：

A1	B1	C1
第一季	1 月	
	2 月	
	3 月	
小计：		
第二季	4 月	
	5 月	
	6 月	
小计：		
第三季	7 月	
	8 月	
	9 月	
小计：		

上主格

单元格横向扩展时，有时候需要跟着被复制的单元格不一定在该单元格的下边，此时，需要对被复制的单元格设置上主格属性。

A 单元格横向扩展时，B 单元格被同步复制，此时 A 单元格称为 B 单元格的上主格。B 单元格称为 A 单元格的附属单元格。上主格扩展时，附属单元格被横向同步复制，且与上主格的相对位置保持不变，附属单元格本身的附属单元格也将被复制。复制时，复制出来的新单元格的所有属性都引用被复制单元格的属性；在实际的报表逻辑中，附属单元格往往是主单元格在同一维上的更细划分，或者是主单元格当前维度的聚集，有时候附属单元格可能是主单元格的标题。

例 1：附属单元格是主单元格的更细划分

假设：B1 是 C1 的上主格，C1 是 D1 的上主格

A1	B1（年）	C1（季度）	D1（月份）
A2	B2		

扩展后：

A1	2000 年	第一 季	1 月	2 月	3 月	第二 季	4 月	5 月	6 月	第三 季	7 月

例 2：附属单元格是主单元格在当前维度的聚集

假设:C1 是 B1 单元格的附属单元格，B1 是 C1 的上主格

A1	B1（季度）	C1（小计）
A2	B2（月份）	
A3	B3	C3

扩展后：

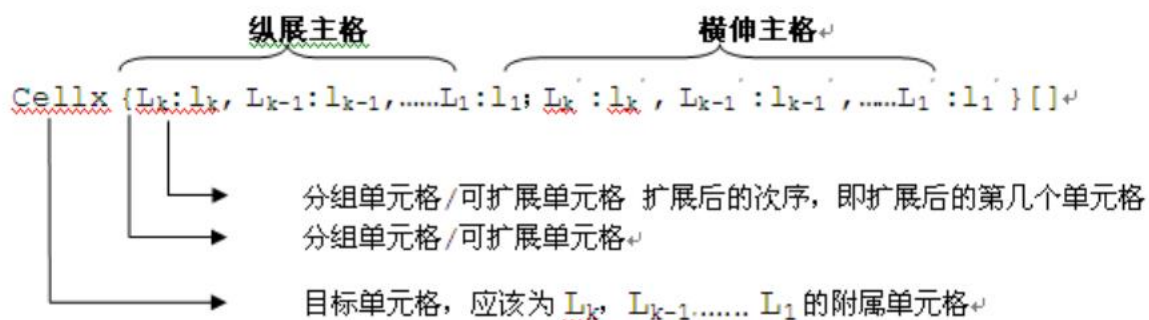
A 1	第一季			小 计	第二季			小 计	第三季			小 计
A 2	1 月	2 月	3 月		4 月	5 月	6 月		7 月	8 月	9 月	
A 3												

单元格的层次坐标

在进行报表设计时，单元格尚未进行扩展，但是单元格的表达式往往需要对扩展后的单元格进行运算。

例如：C2 单元格扩展成了 c2-c10，如果其中一个单元格的表达式希望对扩展后的 c3 和 c4 进行求和，在扩展前，表达式很难描述这样的关系。为避免混淆，应该对扩展后的单元格进行唯一性定义，这就是单元格的层次坐标

表达式规则：



说明:

1. L_k 为 Cellx 的左主格, l_k 为左主格扩展后的次序, 即扩展后的第几个单元格,
如果不指定 l_k 或者 l_k 为 0, 则表示为当前表达式所在单元格所属的当前左
主格 L_k , Cellx 为目标单元格, 应该为 $L_k, L_{k-1}, \dots, L_1$ 的附属单元格

1. 如果没有左主格, 只有上主格的话, 分号不能省略, 即写成 $\text{Cellx} [; L'_k:l'_k, L'_{k-1}:l'_{k-1}, \dots, L'_1:l'_1] \{ \}$
2. 如果没有上主格, 只有左主格的话, 分号可以省略, 即写成 $\text{Cellx} [L_k:l_k, L_{k-1}:l_{k-1}, \dots, L_1:l_1] \{ \}$
3. l_k 的次序是从近到远的, 就是从离当前格最近的主格开始的
4. 表达式最后面跟的花括号表示集合

举例:

假设: A2—A7 单元格由 A2 单元格扩展而来

扩展前:

A1	B1	C1	D1
A2	B2	C2	D2

扩展后：

a1	b1	c1	d1
a2	b2	c2	d2
		c3	d3
	b4	c4	d4
		c5	d5
		c6	d6
a7	b7	c7	d7
		c8	d8
		c9	d9
	b10	c10	d10
		c11	d11
		c12	d12

如果 C2{A2:0,B2:0} 表达式写在 d4 单元格中，那么 d4 就是当前格，B2:0 表示当前格所属的 B2 分组格，也就是扩展后的 b4，A2:0 表示当前格所属的 A2 分组格，也就是扩展后的 a2，因此 C2{A2:0,B2:0} 表达式表示扩展后的 c4-c6 单元格。

C2{A2:2,B2:1} 表达式中，A2:2 表示 A2 扩展后的第二个单元格，即扩展后的 a7，B2:1 表示 a7 分组区域内，B2 扩展后的第一个单元格，即扩展后的 b7，因此整个表达式表示 c7,c8,c9 三个单元格。

如果 B2{A2:0,B2:0} 写在 d11 单元格中，那么 d11 就是当前格，A2:0 表示当前格所属的 A2 分组格，也就是扩展后的 a7，B2:0 表示当前格所属的 B2 分组格，也就是扩展后的 b10，整个表达式表示扩展后的 b10 单元格。

C2[B2:0,A2:0] {}：如果 C2[B2:0,A2:0] {} 表达式写在 d4 单元格中，那么 d4 就是当前格，B2:0 表示当前格所属的 B2 分组格，也就是扩展后的 b4，A2:0 表示当前格所属的 A2 分组格，也就是扩展后的 a2，因此 C2[B2:0,A2:0] {} 表达式表示扩展后的 c4-c6 单 元格。

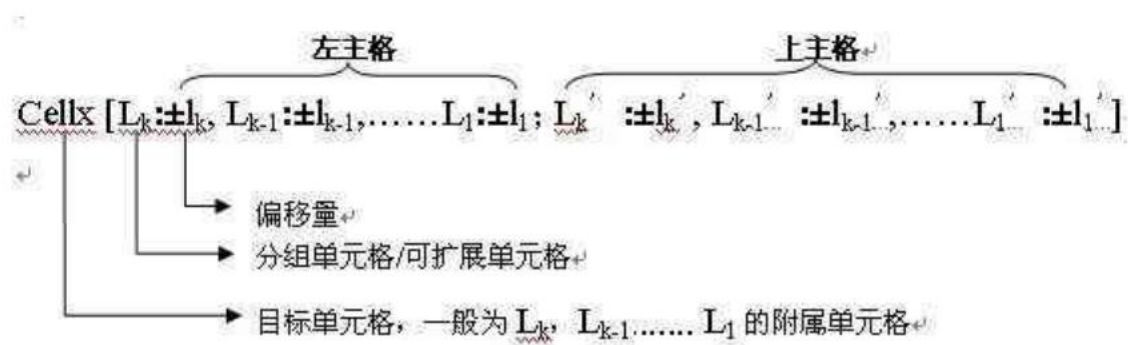
C2[B2:1,A2:2] {}： A2:2 表示 A2 扩展后的第二个单元格，即扩展后的 a7，B2:1 表示 a7 分组区域内，B2 扩展后的第一个单元格，即扩展后的 b7，因此整个表达式表示 c7, c8, c9 三个单元格 。

B2[B2:0,A2:0] {}：如果 B2[B2:0,A2:0] {} 写在 d11 单元格中，那么 d11 就是当前格，A2:0 表示当前格所属的 A2 分 组格，也就是扩展后的 a7，B2:0 表示当前格所属的 B2 分组格，也就是扩展后的 b10，整个表达式表示扩展后的 b10 单 元格 。

单元格的位移表达式

报表中常常需要计算同期比、比上期之类的与时间相关的运算，而这些运算往往需要用到下一行的数据减上一行数据，后一列数据减前一列数据，等等，这种涉及到行间、列间的运算，称为位移运算，相关的表达式称为位移表达式。

表达式规则：



说明:

1. L_k 为 Cellx 的左主格, l_k 为单元格的偏移量, 即当前表达式所在单元格的左主格 L_k 的上 l_k 格或下 l_k 格, 如果不指定 l_k , 则表示为当前所在的左主格 L_k , Cellx 为目标单元格, 一般为 $L_k, L_{k-1}, \dots, L_1$ 的附属单元格
2. 如果没有左主格, 只有上主格的话, 分号不能省略, 即写成 $\text{Cellx} [; L'_k: \pm l'_k, L'_{k-1}: \pm l'_{k-1}, \dots, L'_1: \pm l'_1]$
3. 如果没有上主格, 只有左主格的话, 分号可以省略, 即写成 $\text{Cellx} [L_k: \pm l_k, L_{k-1}: \pm l_{k-1}, \dots, L_1: \pm l_1]$
4. L_k 的次序是从近到远的, 就是从离当前格最近的主格开始的

举例:

扩展前：

A1	B1	C1	D1
A2	B2	C2	D2

扩展后：

a1	b1	c1	d1
a2	b2	c2	d2
		c3	d3
	b4	c4	d4
		c5	d5
		c6	d6
a7	b7	c7	d7
		c8	d8
		c9	d9
	b10	c10	d10
		c11	d11
		c12	d12

如果C2{A2:+1, B2:-1}表达式写在d4单元格，d4单元格就是当前格，A2:+1表示当前格所属A2分组格的下一个，即a7；B2:-1表示当前格所属B2分组格的上一格，当前格所属的B2分组格是b4，位于第二个，他的上一格是第一个，即a7分组格区域里的第一个，也就是b7；所以C2{A2:+1, B2:-1}最终指向c7单元格

对于 d8 单元格，C2{A2,B2}表示 c7 至 c9 三个单元格

对于 d10 单元格，C2{-1}表达式没有意义；对于 d11 单元格，C2{-1}表达式表示 c10

C2[B2, A2]：表示当前格所在当前分组格 B2 和 A2 的共同覆盖区域中的所有 C2 单元格

C2[B2:-1, A2:+1]：表示当前格所在当前分组格 B2 之前偏移一个位置的 B2 分组格和当前格所在当前分组格 A2 之后偏移一个位置的 A2 分组格共同覆盖区域中的所有 C2 单元格

C2[-1]：表示离当前格最近的分组格之前偏移一个位置的分组格覆盖区域中的 C2 单元格

B2[B2, A2]= B2[B2:±0, A2:±0]= B2[B2:0, A2:0]=B2：表示当前的 B2 单元格

函数

数据级函数

select: 从数据集中取出一组数据

语法:

datasetName.select(selectExp{, descExp{, filterExp{, sortExp{}}})

参数说明:

selectExp	选出的字段或其表达式
descExp false 为正序	选出数据的排序规则，true 为逆序，
filterExp	过滤表达式
sortExp	排序依据表达式

group: 根据分组表达式，从数据集中选出一组组集

语法:

datasetName.group(selectExp{, descExp{, filterExp{, sortExp{}}})

参数说明:

selectExp	选出的分组表达式
descExp true 为逆序，false 为顺序	分组表达式结果数据的排序顺序，
filterExp	过滤表达式
sortExp	排序依据表达式

select_one: 从数据集中根据选出字段或表达式以及选出条件，选出一个数据

语法:

datasetName.select_one(selectExp{, filterExp{, filterExp{, filterExp{}}})

参数说明:

selectExp	选出字段或表达式
filterExp	过滤条件

sum: 从数据集中，从满足条件的记录中，算出给定字段或表达式的总和

语法:

```
datasetName.sum(selectExp{,filterExp})
```

参数说明:

selectExp	需求求和的字段或表达式
filterExp	条件表达式

其他函数:

集合函数、日期函数、字符串函数、数学函数等，在此就不罗列了

填报表

能否由用户填写，并将填写内容根据报表的定义，回存到数据库的报表，称之为填报表。

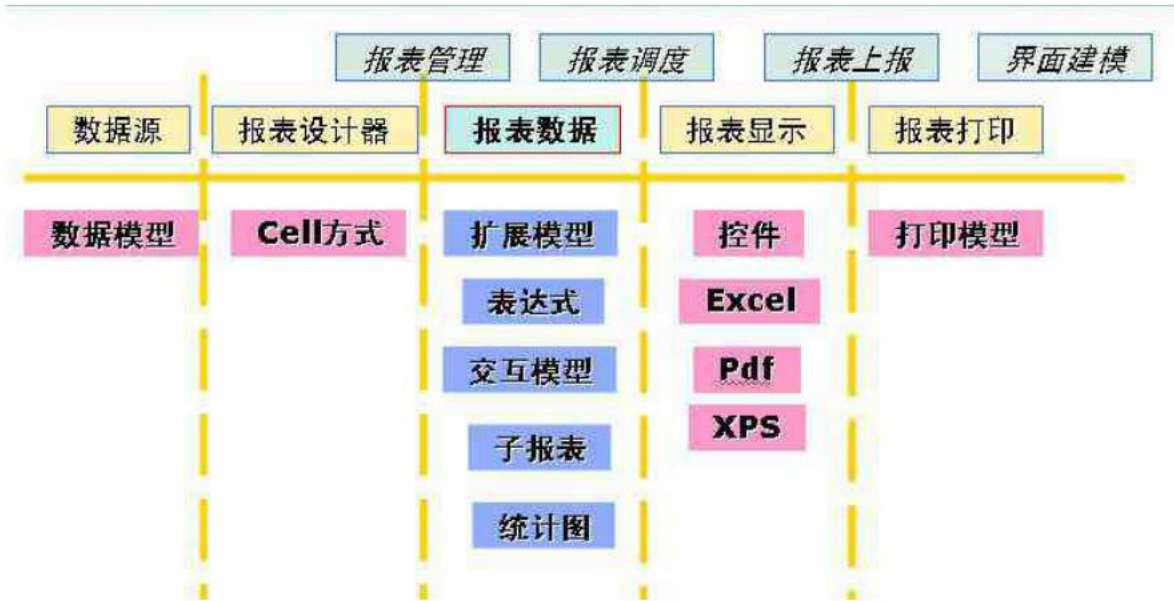
1. 填报表的报表样式、扩展规则和普通报表完全一样
2. 填报表增加了更新属性、自动计算、合法性检查、是否可写、编辑风格等属性

报表模型概述

介绍

本文档作为报表开发的分析文档，对报表分析在功能或者概念上进行描述和细化，不包括报表开发的设计文档

总体组件图



注：报表调度器、管理、上报、界面、分析等为待扩充功能

主要产品组成

报表设计器

图形化 Cell 形式报表设计器，自带报表运算引擎，可预览打印报表，并可生成其它格式保存

功能：

- 1. EXCEL 网格式的表格编辑方案，支持横纵向合并格
- 2. 批量选中修改，剪贴板支持，复制时自动调整公式
- 3. 多次 UNDO/REDO
- 4. 动态更新引用单元格
- 5. 动态显示表达式值

报表服务器

在后台提供统计报表运算处理的服务，开放报表引擎 API 接口

功能：

- 数据源
 1. 同一张报表可有多个数据源，多张报表可以共有一个数据源
 2. 考虑允许适配多个数据源
- 扩展模型
 1. 横纵向对称扩展，自动复制或扩大单元格，方便制作交叉表
 2. 常数序列扩展，如按 1-12 月扩展
 3. 数据表直接扩展，根据表中取出的数据进行扩展
 4. 分组扩展，根据分组结果扩展
 5. 多级层次扩展，可叠加处理，行列方向均可以进行，方便制作多层交叉表
 6. 混合分片，可将固定和变动部分任意组合
- 数据模型
 1. 智能表达式，可根据所处的位置决定运算的范围
 2. 表达式均可设置运算条件
 3. 跨行跨组的运算，行列方向对称，可方便计算同期比、比上期、占比等运算
 4. 支持报表参数
- 单元格属性
 1. 常见的显示格式，如浮点数、串、日期时间等
 2. 代码值与显示值映射
 3. 支持属性表达式，可根据单元格数据决定，如颜色、内容等
 4. 单元格中斜线
 5. 固定图片与图像字段

报表显示控件

功能：

- 输出格式
 1. 支持 Web 的 HTML 显示

2. 可导出 PDF，可同时输出统计图
 3. 可导出 EXCEL，可同时输出报表中的统计图
 4. 带格式的文本
- 预览打印
 - 支持显示不分页数据

架构描述

介绍

本文档是报表平台的技术架构描述，文中会描述主要的设计思路和设计决策。

架构目标和约束

报表平台作为独立的公共服务产品，可以简单的集成到各产品的报表应用

可维护性和可扩展性

- 扩展运算与分页独立实现,代码更易于维护
- 扩充表达式可以提高报表的运算能力
- 在此报表引擎上可以构建多维分析应用

可用性

- 能适用于上万条记录的报表数据量
- 适用于 Web 页面显示

易用性

- 针对技术人员，提供类似 Excel 编辑设计器
- 对多维分析提供专用设计器
- 其他情况下,需要再行开发设计器

实现约束

- 采用 XXX 实现
- 使用与 XXX 相同的表达式引擎和类似的表达式语法
- 采用 XXX 作为显示组件

架构表现

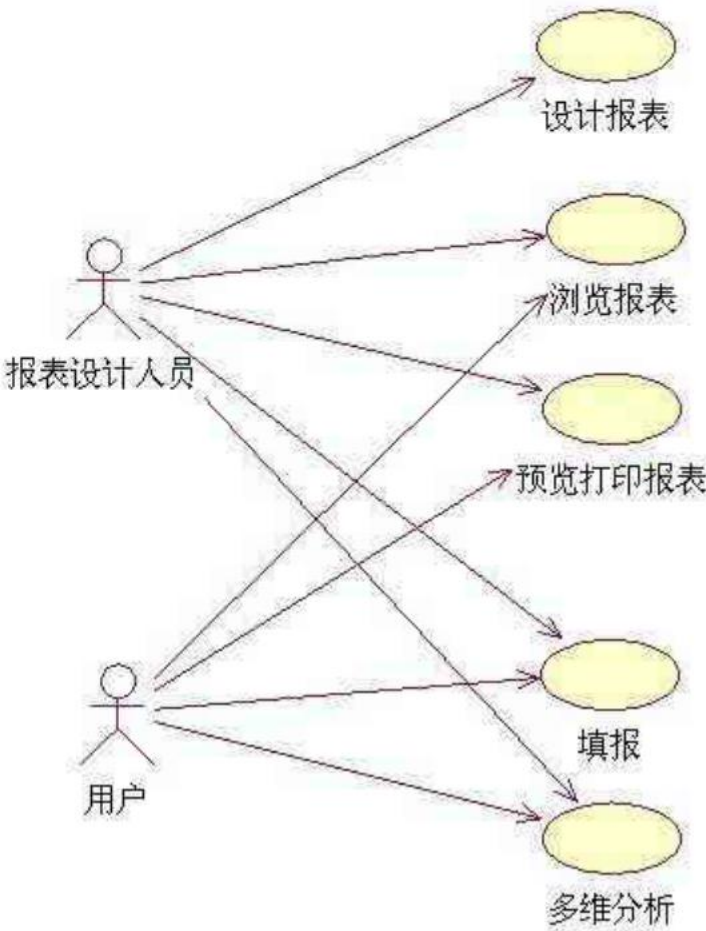
文档描述了系统的主要模型，包括：

- [用例模型](#): 从系统外部角度表现重要的参与者和用例
- [分析模型](#): 以平台独立性提炼用例重要分析元素和用例的分析实现
- [设计模型](#): 从平台方面描述部署结构、设计元素结构等

用例模型从外部系统的角度描述了系统的行为。

用例模型

- 应用用例：描述用户如何与系统交互以实现预期的功能

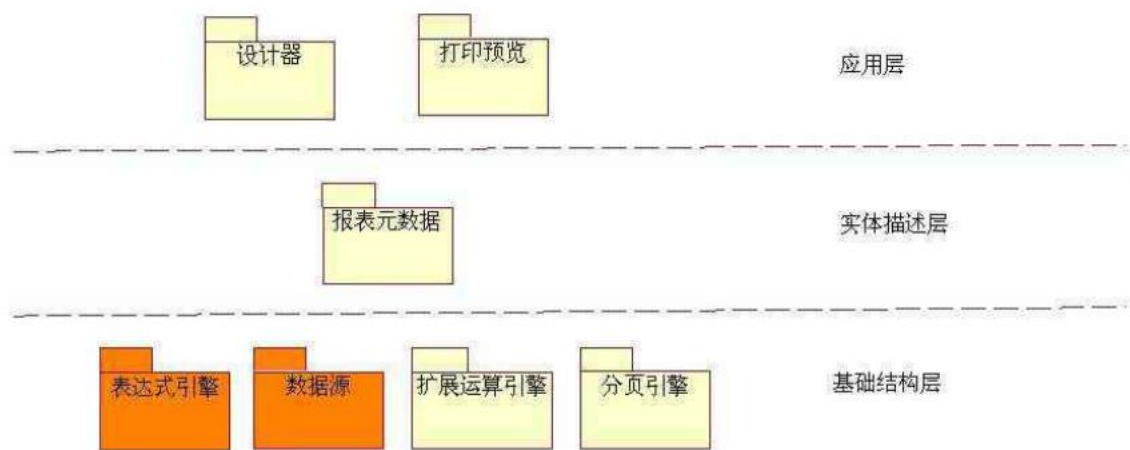


用例	描述	基本要
设计报表	报表设计人员通过这个用例设计一张报表	通过提供的报表设计器进行报表定义
浏览报表	使用者通过此用例查看报表数据	查看前有些报表需要输入参数
预览打印报表	使用者通过此用例以分页方式查看报表, 并可以打印报表	
填报	使用者通过此用例进行业务数据的输入	对于上报可能需要自动汇总功能
多维分析	数据分析人员通过此用例可以从多维度分析数据	专有的多维报表设计器, 查看报表时提供交互

- 基础结构用例：描述应用用例的每一步如何添加诸如可用性、可靠性、性能、容错性等质量属性
- 扩展用例

分析模型

架构性重要分析元素

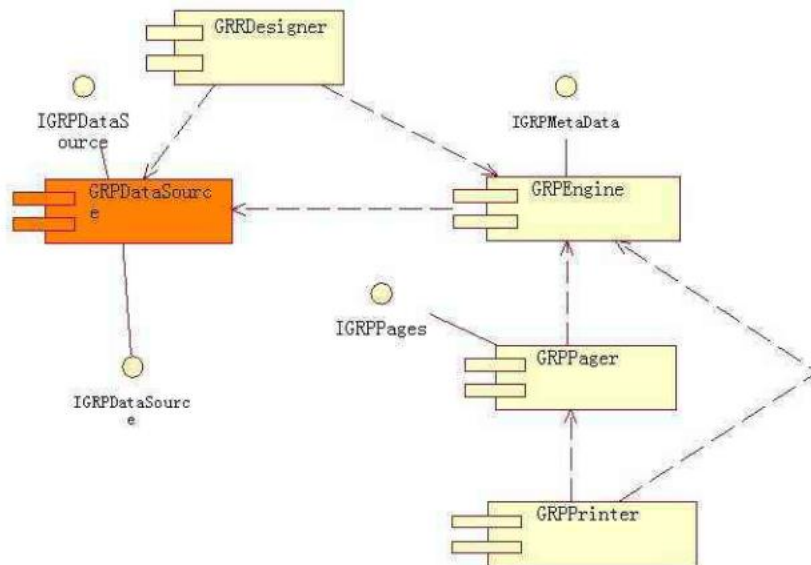


包	描述	基本要求
扩展运算引擎	对设计的报表通过模型扩展并进行运算, 生成完整的最终报表	需要注意扩展和运算的效率
分页引擎	把最终报表通过分页引擎和分页规则生成多页报表	
数据源	提供给报表的数据来源	外部定义数据源程序集, 实现特定接口
表达式引擎	实现报表运算中定义的表达式	可以扩充表达式增加报表的运算能力
报表元数据	报表的定义描述文件	报表文件以 XML 方式存储
设计器	对报表提供统一的表格式设计器, 对多维分析提供专有设计器	使用 ViewGrid 控件
打印预览	以分页方式浏览数据	实现轻量级预览控件, 减少系统资源

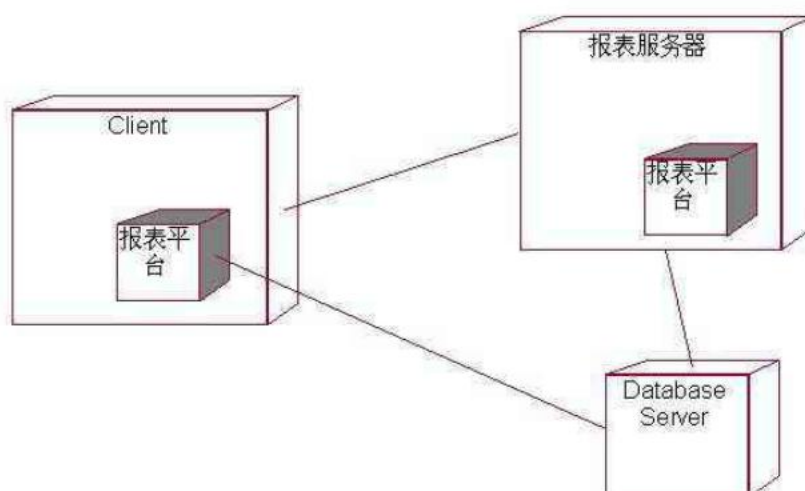
架构性重要用例实现

设计模型

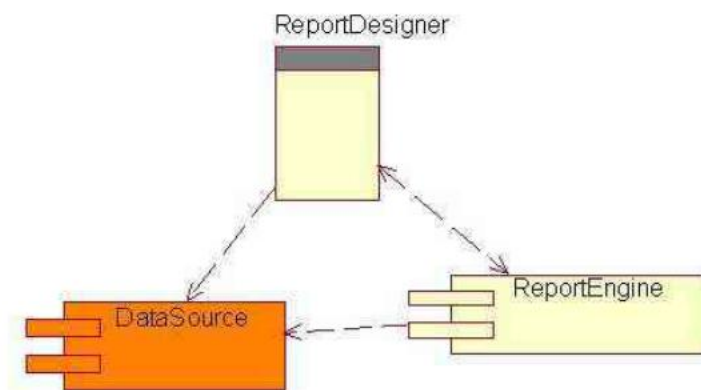
设计元素



部署结构



组件结构



扩展模型

介绍

本文档是参考润乾报表模型，继《[报表应用思路](#)》的进一步分析，说明平台报表中涉及的多个相关模型，为后续报表开发提供概念以及总体设计支持

术语

名称	说明
单元格	报表由行列整齐的格子组成，这些格子我们称为单元格，所有的单元格组成了报表。
主格和附属格	主格和附属格是互相关联的，当 A 单元格扩展时，B 单元格被 同步复制，此时 A 单元格称为 B 单元格的主格，B 单元格称为 A 单元格的附属单元格。
横伸主格	如果 A 单元格横向扩展则 A 为横伸主格
纵展主格	如果 A 单元格纵向扩展则 A 为纵展主格
合并单元格	多个连续的单元格通过合并操作后显示为一个大的格子，合并后的格子成为合并单元格
单元格属性值和表达式	单元格的大部分属性都有属性值和表达式两种。 属性值是在编辑报表时确定的，在运行期间不必经过运算，而表达式在运行期间需要经过运算而获得属性值。
单元格的层次坐标	通过单元格的坐标表示法唯一标识扩展后的格子

扩展概念

可扩展单元格

当单元格的数据值表达式包含[集合函数](#)时，该单元格默认为可扩展单元格,可以修改扩展方向属性为默认、[横向扩展](#)或者[纵向扩展](#)。对于不包含集合函数的单元格，扩展方向属性默认为不可扩展。

横向扩展

描述：

1. 当单元格为可扩展单元格时，可以把该单元格的扩展方向设为横向扩展，此时该单元格会横向进行复制，复制单元格的数据值依次为集合函数的结果数据值，集合返回几个值，单元格就复制几个，这种现象称为[单元格的横向扩展](#)

2. 单元格进行扩展时，主单元格或同级别单元格被拉大成[合并单元格](#)，附属单元格被复制（参考[主格和附属格的认定规则](#)）
3. 复制出来的新单元格的所有属性都引用被复制单元格的属性

例 1：横向扩展

从下图中可以看出，B2 单元格是横向扩展格，我们把 B2 单元格边框用颜色标识一下，以便扩展后容易识别，如下图所示

扩展前：

	B1	C1
	B2	C2
	B3	C3

扩展后：

	B1				F1
	B2				F2
	B3				F3

说明：在这个例子中，B2 为横向扩展格，因此 B3 缺省为 B2 的附属单元格，因此当 B2 扩展时，B3 被复制；B1 虽然在 B2 的上方，但由于 B1 不是扩展格，所以不是 B2 的横伸主格，[B1 和 B2 的横伸主格都是 00 格](#)，因此 B1 和 B2 属于同一级别的单元格，因此 B2 扩展的时候，也会把同级别单元格 B1 拉大。

例 2：下方较高级别或同级别单元格被横向拉大

扩展前：假设 B2 是 B1 的附属单元格，B4 也是 B1 的附属单元格，因此从逻辑上讲，B4 至少和 B2 平级，当 B2 进行扩展时，B4 应该延展，不应该复制

	B1	C1
	B2	C2
	B3	C3
	B4	C4

扩展后：

	B1				F1
	B2				F2
	B3				F3
	B4				F4

纵向扩展

描述：

1. 当单元格为可扩展单元格时，可以把该单元格的扩展方向设为纵向扩展，此时该单元格会纵向进行复制，复制单元格的数据值依次为表达式的结果数据值，表达式返回几个值，单元格就复制几个，这种现象称为**单元格的纵向扩展**
2. 单元格进行扩展时，主单元格或同级别单元格被拉大成**合并单元格**，附属单元格被复制（参考 [主格和附属格的认定规则](#)）
3. 复制出来的新单元格的所有属性都引用被复制单元格的属性

例 1：纵向扩展

从下图中可以看出，B2 单元格是纵向扩展格，我们把 B2 单元格边框用颜色标识一下，以便扩展后容易识别，如下图所示， 扩展前：

A1	B1	C1
A2	B2	C2
A3	B3	C3

扩展后：

A1	B1	C1
A2	B2	C2
A5	B5	C5

说明：在这个例子中，B2 为纵向扩展格，因此 C2 缺省附属于 B2，因此当 B2 纵向扩展时，C2 被复制；A2 虽然在 B2 的左边，但 A2 不是扩展格，所以 A2 不是 B2 的 纵展主格，A2 和 B2 的纵展主格都是 00 格，因此 A2 和 B2 属于同一级别的单元格，所以 B2 扩展的时候，也会把同级别单元格 A2 拉大。

例 2：右边较高级别或同级别单元格延展

扩展前：假设 B2 是 A2 的附属单元格，D2 也是 A2 的附属单元格，因此从逻辑上讲，D2 至少和 B2 平级，当 B2 进行扩展时，D2 应该延展，不应该复制

A1	B1	C1	D1
A2	B2	C2	D2
A3	B3	C3	D3

扩展后：

A1	B1	C1	D1
A2	B2	C2	D2
A5	B5	C5	D5

说明：由于把 A2 设成了纵向扩展格，因此 A2 是 B2 的纵展主格，同时 C2 的纵展主格设成了 A2，因此 B2 和 C2 都附属于 A2，他们属于同一级别的单元格，因此 B2 扩展时，同级别单元格 C2 被拉大。

不可扩展

当单元格里表达式的值是单值时，该单元格默认为不可扩展的单元格。
不可扩展的单元格不能缺省做主格。

默认

- 1. 单元格的表达式为单值表达式时，该单元格默认为不可扩展单元格
- 2. 单元格的表达式为集合表达式时，该单元格默认为可扩展单元格 （参考 [主格和附属格的认定规则](#)）
 - 1. 可扩展单元格的横伸主格横向扩展时，该单元格默认为横向扩展
 - 2. 可扩展单元格的纵展主格纵向扩展时，该单元格默认为纵向扩展
 - 3. 可扩展单元格的主格为行首格时默认为横向扩展，为列首格时该单元格默认为纵向扩展
 - 4. 不满足前面条件则默认为纵向扩展

横伸主格

描述：

- 1. A 单元格横向扩展时，B 单元格被同步复制，此时 A 单元格称为 B 单元格的 横伸主格，B 单元格称为 A 单元格的附属单元格
- 2. 横伸主格扩展时，附属单元格被横向同步复制，且与横伸主格的相对位置保持不变，附属单元格本身的附属单元格也将被复制。复制时，复制出来的新单元格的所有属性都引用被复制单元格的属性
- 3. 在实际的报表逻辑中，附属单元格往往是主单元格在同一维上的更细划分，或者是主单元格当前维度的聚集，有时候附属单元格可能是主单元格的标题。
- 4. 单元格横向扩展时，有时候需要跟着被复制的附属格不一定在该单元格的下边，此时，需要对附属单元格设置横伸主格属性。

例 1：附属单元格是主单元格的更细划分

假设：B1 是 C1 的 横伸主格，C1 是 D1 的横伸主格

A1	B1（年）	C1（季度）	D1（月份）
A2	B2		

扩展后：

A1	2000	第一	1 月	2 月	3 月	第二	4 月	5 月	6 月	第三	7 月
----	------	----	-----	-----	-----	----	-----	-----	-----	----	-----

	年	季				季				季	

例 2：附属单元格是主单元格在当前维度的聚集

假设:C1 是 B1 单元格的附属单元格，B1 是 C1 的横伸主格

A1	B1（季度）	C1（小计）
A2	B2（月份）	
A3	B3	C3

扩展后：

A 1	第一季			小 计	第二季			小 计	第三季			小 计
A 2	1 月	2 月	3 月		4 月	5 月	6 月		7 月	8 月	9 月	
A 3												

纵展主格

描述：

1. A 单元格纵向扩展时，B 单元格被同步复制，此时 A 单元格称为 B 单元格的 纵展主格。B 单元格称为 A 单元格的附属单元格。
2. 纵展主格扩展时，附属单元格被纵向同步复制，且与纵展主格的相对位置保持不变，附属单元格本身的附属单元格也将被复制。复制时，复制出来的新单元格的所有属性都引用被复制单元格的属性
3. 在实际的报表逻辑中，附属单元格往往是主单元格在同一维上的更细划分，或者是主单元格当前维度的聚集，有时候附属单元格可能是主单元格的标题。
4. 单元格纵向扩展时，有时候需要跟着被复制的单元格不一定在该单元格的右边，此时，需要对被复制的单元格设置纵展主格属性。

例 1：附属单元格是主单元格的更细划分

假设：A2 是 A3 的 纵展主格，A3 是 A4 的纵展主格

A1	B1
A2（年）	B2
A3（季度）	B3
A4（月份）	B4

扩展后：

A1	B1
2000 年	
第一季	
1 月	
2 月	
3 月	
第二季	
4 月	
5 月	
6 月	
第三季	
7 月	

例 2：附属单元格是主单元格的在当前维度的聚集

假设 A3 是 A2 单元格的附属单元格，A2 是 A3 的纵展主格

A1	B1	C1
A2（季度）	B2（月份）	C2
A3（小计）		C3

扩展后：

A1	B1	C1
第一季	1 月	
	2 月	
	3 月	
小计：		
第二季	4 月	
	5 月	
	6 月	
小计：		
第三季	7 月	

	8 月	
	9 月	
小计:		

主单元格和附属单元格

规则

扩展规则

单元格进行扩展时，主单元格或同级别单元格拉大成[合并单元格](#)，附属单元格被复制（横伸主格扩展时，附属格被横向同步复制； 纵展主格扩展时，附属格被纵向同步复制）

引用规则

单元格扩展时，附属单元格可以动态引用主单元格的值。随着主单元格扩展，附属单元格被复制到不同的位置，对主单元格的引用也相应的变化。

例：`ds1.select(col1,col2=A2)`（其中 A2 为扩展主格）

统计规则

单元格扩展时，附属单元格如采用缺省的聚集表达式，则缺省统计所属主格区域内所有目标单元格的值 例：`Sum(C3{})`

主格和附属格认定规则

横伸主格的认定： 横伸主格的认定是向上依次查找。这里假设当前格是 B3，上方单元格是 B2

1. 当前单元格 **B3** 设置了横伸主格属性，则横伸主格为设定的值
2. 如果没有设定横伸主格属性，则检查上方单元格 **B2** 是否是横向扩展格并且在列范围内包含了当前单元格，如果是，上方单元格 **B2** 就是当前格 **B3** 的横伸主格。
3. 如果 **B2** 不是横向扩展格，就检查 **B2** 的横伸主格，**B3** 的横伸主格跟随 **B2**，即 **B2** 和 **B3** 有共同的横伸主格。
4. **B2** 的横伸主格的认定方法，从步骤 1 开始执行
5. 如果执行到首行仍未有横伸主格，则横伸主格缺省为 **00** 格

纵展主格的认定：纵展主格的认定是向左依次查找。这里假设左边单元格是 B2，当前格是 C2

- 1. 当前单元格 C2 设置了纵展主格属性，则纵展主格为设定的值
- 2. 如果没有设定纵展主格属性，则检查左边单元格 B2 是否是纵向扩展格并且在行范围内包含了当前单元格，如果是，左边单元格 B2 就是当前格 C2 的纵展主格
- 3. 如果 B2 不是纵向扩展格，就检查 B2 的纵展主格，C2 的纵展主格跟随 B2，即 B2 和 C2 有共同的纵展主格
- 4. B2 的纵展主格的认定方法，从步骤 1 开始执行
- 5. 如果执行到最左边仍未有纵展主格，则纵展主格缺省为 00 格

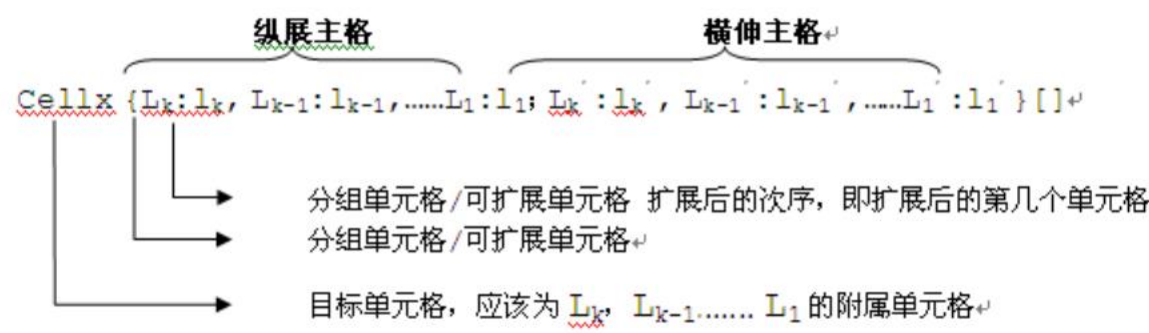
单元格的层次坐标和汇总运算

绝对层次坐标

描述：

- 1. 在进行报表设计时，单元格尚未扩展，但是单元格的表达式往往需要对扩展后的单元格进行运算
- 2. 例如：C2 单元格扩展成了 c2-c10，如果其中一个单元格的表达式希望对扩展后的 c3 和 c4 进行求和，在扩展前，表达式很难描述这样的关系。 为避免混淆，应该对扩展后的单元格进行唯一性定义，这就是单元格的层次坐标

表达式规则



说明：

- 1、 L_k 为 $Cellx$ 的纵展主格， lk 为纵展主格扩展后的次序，即扩展后的第几个单元格，如果不指定 lk 或者 lk 为 0，则表示为当前表达式所在单元格所属的当前纵展主格 L_k ， $Cellx$ 为目标单元格，应该为 $L_k, L_{k-1}, \dots, L_1$ 的附属单元格
- 2、如果没有纵展主格，只有横伸主格的话，分号不能省略，即写成 $Cellx \{; L_k':lk', L_{k-1}':lk-1', \dots, L_1':l1' \} []$
- 3、如果没有横伸主格，只有纵展主格的话，分号可以省略，即写成 $Cellx \{L_k:lk, L_{k-1}:lk-1, \dots, L_1:l1 \} []$
- 4、 L_k 的次序是从远到近的，就是从离当前格最远的主格开始的
- 5、表达式最后面跟的花括号表示集合

举例：

假设：a2—a7 单元格由 A2 单元格扩展而来

扩展前：

A1	B1	C1	D1
A2	B2	C2	D2

扩展后：

a1	b1	c1	d1
a2	b2	c2	d2
		c3	d3
	b4	c4	d4
		c5	d5
		c6	d6
a7	b7	c7	d7
		c8	d8
		c9	d9
	b10	c10	d10
		c11	d11
		c12	d12

如果 C2{A2:0,B2:0}[] 表达式写在 d4 单元格中, 那么 d4 就是当前格, B2:0 表示当前格所属的 B2 分组格, 也就是扩展后的 b4, A2:0 表示当前格所属的 A2 分组格, 也就是扩展后的 a2, 因此 C2{A2:0,B2:0}[] 表达式表示扩展后的 c4-c6 单元格

C2{A2:2,B2:1}[] 表达式中, A2:2 表示 A2 扩展后的第二个单元格, 即扩展后的 a7, B2:1 表示 a7 分组区域内, B2 扩展后的第一个单元格, 即扩展后的 b7, 因此整个表达式表示 c7,c8,c9 三个单元格

如果 B2{A2:0,B2:0}[] 写在 d11 单元格中, 那么 d11 就是当前格, A2:0 表示当前格所属的 A2 分组格, 也就是扩展后的 a7, B2:0 表示当前格所属的 B2 分组格, 也就是扩展后的 b10, 整个表达式表示扩展后的 b10 单元格

C2{A2:0,B2:0}[]: 如果 C2{A2:0,B2:0}[] 表达式写在 d4 单元格中, 那么 d4 就是当前格, B2:0 表示当前格所属的 B2 分组格, 也就是扩展后的 b4, A2:0 表示当前格所属的 A2 分组格, 也就是扩展后的 a2, 因此 C2{A2:0,B2:0}[] 表达式表示扩展后的 c4-c6 单元格

C2{A2:2,B2:1}[]: A2:2 表示 A2 扩展后的第二个单元格, 即扩展后的 a7, B2:1 表示 a7 分组区域内, B2 扩展后的第一个单元格, 即扩展后的 b7, 因此整个表达式表示 c7,c8,c9 三个单元格

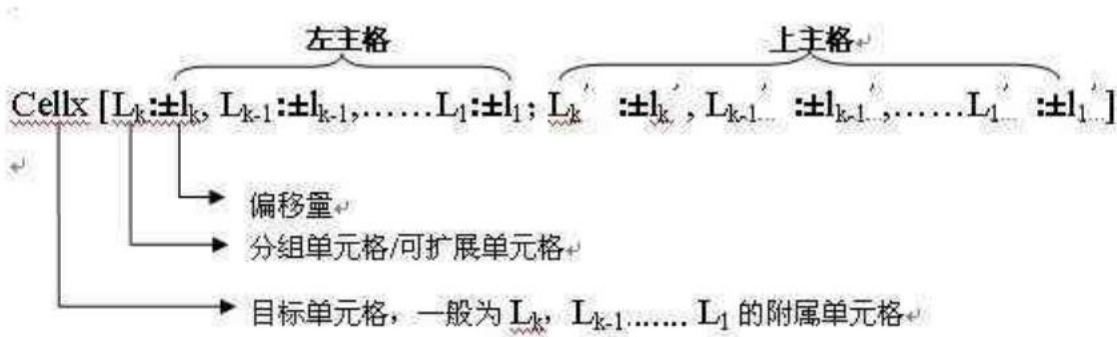
B2{A2:0,B2:0}[]: 如果 B2{A2:0,B2:0}[] 写在 d11 单元格中, 那么 d11 就是当前格, A2:0 表示当前格所属的 A2 分组格, 也就是扩展后的 a7, B2:0 表示当前格所属的 B2 分组格, 也就是扩展后的 b10, 整个表达式表示扩展后的 b10 单元格

相对层次坐标（位移坐标）

描述：

- 1. 报表中常常需要计算同期比、比上期之类的与时间相关的运算，而这些运算往往需要用到下一行的数据减上一行数据，后一列数据减前一列数据，这种涉及到行间、列间的运算，称为位移运算，相关的表达式称为位移表达式

表达式规则



说明：

- 1. L_k 为 Cellx 的 纵展主格，l_k 为单元格的偏移量，即当前表达式所在单元格的纵展主格 L_k 的上 l_k 格或下 l_k 格，如果不指定 l_k，则表示为当前所在的纵展主格 L_k，Cellx 为目标单元格，一般为 L_k, L_{k-1}, L₁ 的附属单元格
- 2. 如果没有纵展主格，只有横伸主格的话，分号不能省略，即写成 Cellx [; L'_k:±l'_k, L'_{k-1}:±l'_{k-1}, L'₁:±l'₁]
- 3. 如果没有横伸主格，只有纵展主格的话，分号可以省略，即写成 Cellx [L_k:±l_k, L_{k-1}:±l_{k-1}, L₁:±l₁]
- 4. L_k 的次序是从远 到近的，就是从离当前格最远的主格开始的

5. 表达式最后面跟的花括号表示集合

举例：

$C2\{A2, B2\}$ ：表示当前格所在当前分组格 B2 和 A2 的共同覆盖区域中的所有 C2 单元格

$C2\{A2:+1, B2:-1\}$ ：表示当前格所在当前分组格 B2 之前偏移一个位置的 B2 分组格和当前格所在当前分组格 A2 之后偏移一个位置的 A2 分组格共同覆盖区域中的所有 C2 单元格

$C2\{-1\}$ ：表示离当前格最近的分组格之前偏移一个位置的分组格覆盖区域中的 C2 单元格

$B2\{A2, B2\} = B2\{A2:\pm 0, B2:\pm 0\} = B2\{A2:0, B2:0\} = B2$ ：表示当前的 B2 单元格

条件表达式

描述：

单元格的表达式常常需要用到条件表达式，例如：需要获得分组格的值（月份）为 1 月的所有单元格集合

表达式规则

$Cell\{\text{单元格的层次坐标或位移坐标}\} [EXP]$

说明：

1. 所有条件表达式用中括号括起来
2. 条件表达式的返回值为 true 或 false
3. 中括号里为空时表示当前层次坐标下的所有目标单元格的集合

4. 中括号里存在条件表达式，则表示当前层次坐标下，所有符合条件的目标单元格的集合

举例：

数据集定义为： `ds1=select id, 语文, 数学 from 学科` , 示例需要列举出不及格的平均分数

扩展前：

A1:ds1.id[] 纵向扩展	B1:ds1. 语文	C1:ds1. 数学
A2:不及格平均值	B2:avg(B1[B1<60]) ###.0	C2:avg(C1[C1<60]) ###.0

扩展后：

1	40	40
2	50	50
3	60	60
4	70	70
5	80	80
不及格平均值	45.0	45.0

& 运算符

描述：

每个单元格都有自己的层次坐标，用&运算符可以获得单元格位于某个主格中的位置

表达式规则：

`&Cellx` --> 当前单元格的某主格

返回值： 当前单元格位于该主格的位置

举例：

假设：a2, a7 单元格由 A2 单元格扩展而来

扩展前：

A1	B1	C1	D1
A2	B2	C2	D2
A3	B3	C3	D3
A4	B4	C4	D4

扩展后：

a1	b1	c1	d1
a2	b2	c2	d2
		c3	d3
	b4	c4	d4
		c5	d5
		c6	d6
		c7	d7
a7	b7	c8	d8
		c9	d9
		c10	d10
	b10	c11	d11
		c12	d12
		c13	d13

那么：

1. 对于 **d4 单元格**，&A2 返回值为 1，&B2 返回值为 2，&C2 返回值为 1
2. 对于 **d12 单元格**，&A2 返回值为 2，&B2 返回值为 2，&C2 返回值为 3

? 运算符

描述：

在单元格的表达式中，往往需要引用单元格的某主格，当目标单元格的主格和当前单元格的主格在扩展前相同时，表达式中容易引起混淆，\$运算符可以解决这一问题

表达式规则：

?Cellx --> 当前表达式中指定设计单元格的扩展后的单元格的某主格

返回值： 返回当前写表达式单元格的指定主格的数据值

注意：如果不写\$，直接写 Cellx，则表示目标单元格的某主格

举例：

=B1 {?A1=A1-3} 该表达式的含义是：当前格所属的 A1 主格的值等于目标单元格（B1）所属的 A1 格的值减 3

根坐标表示法

描述：

在单元格的表达式中，往往需要引用某个单元格扩展后的所有单元格的值，例如算汇总、算平均等，而当前单元格的主单元格往往不是 00 格，因此需要在层次坐标中有一个根坐标的表示法，表示 00 格下的所有目标单元格

表达式规则：

Cellx{00} --> 目标单元格

返回值：返回 00 格下所有目标单元格的集合

举例

=C2/sum(C2{00}{}) 该表达式的含义是：当前格所属的主格下的 C2 单元格除以所有 c2 单元格的汇总，是一个典型的占比算法

命名规则

1. 合并单元格是以左上角的单元格编号作为合并格的编号的
2. 单元格的命名方式和 excel 类似，列上通过 A,B,C,D,.....来命名每一列，行上通过 1,2,3,4,.....来命名每一行，单元格通过[列名+行名] 来命名，如 A 1

其他规则

1. 合并单元格的某一边边框都是一致的，不能分段做不同设置。
2. 属性值和表达式之间互斥，定义了属性值，就不必定义表达式，定义了表达式，就不必定义属性值。在属性表达式中可以用"@value"来引用单元格的数据值。
3. 设置当前单元格中是否加入斜线，以及斜线的线宽。一个单元格斜线最多可以两条，此时必须把单元格的数据值用逗号分隔

数据模型

介绍

本文档是报表模型的数据模型部分，说明平台报表中涉及的报表数据相关

术语

名称	说明
数据源	数据源是与数据存储的连接。支持数据源适配
数据集	定义报表数据来源的一个二维表
参数	报表往往有参数，例如，日报表，当我们希望生成一张日报表时，首先需要向报表传递日期参数，然后报表才会根据我们传递进去的日期参数，生成该日的日报表。
当前单元格	在进行单元格计算时，写有表达式的单元格为当前单元格。条件表达式中有?单元格表达式都是以当前单元格为基准计算
目标单元格	表达式中返回单元格时，返回的单元格为目标单元格。条件表达式中单元格表达式都是以目标单元格为基准计算
参数	报表往往有参数，例如，日报表，当我们希望生成一张日报表时，首先需要向报表传递日期参数，然后报表才会根据我们传递进去的日期参数，生成该日的日报表。 参数有以下几个特征： 1、 参数有数据类型 2、 参数有缺省值 3、 参数有输入格式和最大宽度 参数可以在数据集和表达式中被引用，表达式中直接写参数名引用，
宏	宏代表没有数据类型的一个字符串，在报表运算时将用宏值替换宏变量所占的位置。可以在报表中任何位置引用宏变量，如单元格值、表达式、数据集定义、单元格属性表达式中等，引用方法为%{宏变量名}
表达式计算阶段	表达式分为在扩展期间和分页期间计算，和页眉相关的表达式在分页阶段计算，如页码、页数等。系统会对自动判断表达式的计算阶段

操作符

符号	说明	左操作数	右操作数	运算结果	示例说明
:	到	单元格	单元格	List 集合	A1:D5 由 A1 和 D5 为两个对角点圈起的矩形中的单元格集合，可以使用在 sum, max 等聚集函数中作参数
in	包含	数值	List 集	布尔值	A1 in (1, 2, 3) 是否在集合中左右操作数

		字符串	合		均可为单值或数组，如 1 in A1{00} [] 表示 1 是否存在 00 格下 A1 的扩展格子的值列表中
to	到	整数	整数	List 集合	1 to 5 表示数组, 等同于 [1, 2, 3, 4, 5]
.	对象成员	数据表	字段	字段值	RKD, RKDID 表示表入库单 ID 字段
?	参数表达式	单元格		当前单元格值	单元格条件过滤时，用参数表达式来获得当前单元格的值。
@	系统宏				@value 表示当前单元格值，@row, @col 表示当前单元格行列号，@DRow, @DCol 表示设计期单元格行列，@pageIndex 代表当前页码，@pageCount 代表总页数
	表示日期类型，@为字符串第一个子目，其余为日期字符串	日期		日期	'@2000-1-1': 表示 2000 年 1 月 1 日
{}	选择条件表达式	字符串			C5 {A5:1} 层次坐标中的条件表达式 C5 {A5:1} [] 表示该层次坐标中的所有目标单元格，代表一个集合
[]	集合	过滤条件以及排序或者数组位置			[1, 2, 3, 4]: 表示一个元素为 1, 2, 3, 4 的数组 CKD. CLID [DJ>10]: 表示材料出库单的单价大于 10 的 CLID 字段列表 CKD. CLID [DJ>10 CLMC]: 表示材料出库单的单价大于 10 的 CLID 字段列表, 列表根据材料名称排序。只有数据集支持排序, 单元格目前不支持。 CKD. CLID [CLMC ACS; CLJG DESC]: 表示材料出库单的所有 CLID 字段列表, 列表根据材料名称顺序, 材料价格倒序 A1 [2] 表示集合单元格 A1 中的第 2 个元素。CKD. CLID [2] 取出库单的第二行材料 ID 值 C5 [C5>60] [2] 标识及格分数列表的第 2 个元素
&	取位置	主单元格		&A1	操作数必须为当前单元格的主格子，位置从 1 开始
%	引用参数	参数标识		参数数据类型	在元数据定义参数列表，%year 表示外面传入的年份参数

集合函数

其他

表格运算示例

当前单元格	表达式	应用场景	备注
B1	A1+10	同级别单元格赋值，比如 A1 为表的一个字段，把 B1 设置成 A1 的值加 10	A1/B1 为同级单元格
	&A1	位置：序号	A1 为扩展单元格
	dsl.id[type=B1]	数据级函数的选择条件，取 type 字段=B1 的所有记录的 ID 字段	
B2	avg(B1[B1<60])	条件表达式，求不及格的平均分	B1 为成绩
C3	C2/sum(C2{00}[])	根坐标：占比运算	计算当前 C2 格占 C2 合计的百分比
	Count(b2{00}[b2>\$b2])+1	排名	B2 为数值单元格
	D2/D2{A2:+0}[(?c2-1)==C2 and ?b2=B2) or ((?b2-1)=B2 and ?c2=4 and C2=10)]	比上期	
	D2/D2{B2:-1,C2:+0}	比去年同期	
	C2+D2[-1]	累计金额	C2 为合同金额，D2 为累计金额

约束

页面宏

- 1. 不能影响扩展过程：不能在集合表达式中使用

页统计

- 1. 页统计表达式只统计单元格集合表达式

算法说明

单元格变量

1. 如果没有设置左主格和上主格，先查找目标单元格和当前单元格共同的左主格 **l** 和上主格 **t**
2. 如果左主格的源格为目标单元格，则返回左主格 **l**；如果上主格的源格为目标单元格，则返回上主格 **t**
3. 如果左主格的源格 **L** 为目标单元格的直接左主格，在 **l** 单元格的附属格中查找与目标单元格匹配的单元格 **c**，如果单元格的上主格属于 **t**，则返回单元格 **c**；上主格按左主格方法查找
4. 如果左主格源格 **L** 是目标单元格的间接左主格，在 **l** 单元格的附属格中查找是目标单元格的递归左主格的单元格 **c**，然后把左主格设置为 **c** 再次调用步骤 4 直到满足条件 3 而调用 3 获取单元格值

单元格列表

1. 查找目标单元格和当前单元格共同的左主格 **l** 和上主格 **t**
2. 如果左主格的源格为目标单元格，则返回左主格 **l**；如果上主格的源格为目标单元格，则返回上主格 **t**
3. 如果左主格的源格 **L** 为目标单元格的直接左主格，在 **l** 单元格的附属格中查找与目标单元格匹配的单元格 **c**，如果单元格不是扩展格，并且上主格属于 **t**，则返回单元格 **c**，如果单元格 **c** 为扩展格，则查找 **c** 的 **ExtCells** 中的上主格属于 **t** 的单元格列表；如果没有符合条件的单元格，则按照 3 从上主格开始查找 并返回符合条件的单元格
4. 如果左主格源格 **L** 是目标单元格的间接左主格，在 **l** 单元格的附属格中查找是目标单元格的递归左主格的单元格 **c**，然后把左主格设置为 **c** 再次调用步骤 4 直到满足条件 3 而调用 3 获取单元格列表

图形模型

介绍

本文档是报表模型的图形支持模型部分，说明平台报表中现在对图形的支持。

术语

名称	说明
图形	对报表模型中支持的图形相关类型的总称
矢量图	公司自己开发的矢量图库，应用在 GCL 项目组中
统计图	对数据进行统计分析的图形，应用在 GCM 项目管理软件中
统计类型	统计图显示类型，目前支持：折线图, 纵向柱形图, 横向柱形图, 面积图, 散点图, 饼图,快速折线图,横向折线图,横向面积图
x 轴	显示的横轴
y 轴	显示代表值的纵轴
z 轴	系列轴
图片	常用图形格式的文件，例如 JPEG、Bitmap 等
页面背景图	报表预览时，在每一页背景中显示的图片
单元格背景图	每个单元格背景中显示的图片
图形样式	图形显示的样式，包括三种类型：None、拉伸、平铺

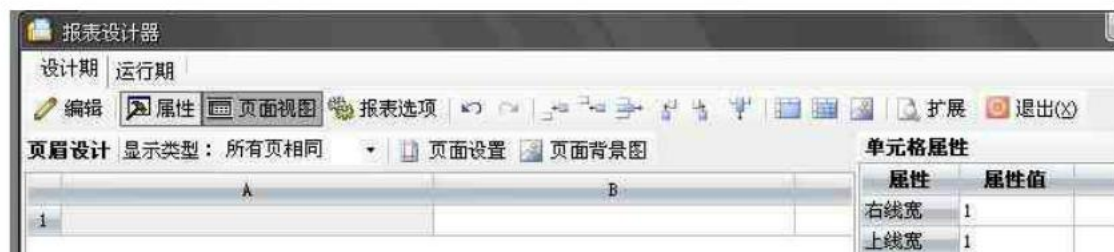
矢量图

图片

报表支持页面背景图和单元格背景图以及单元格图片内容。

页面背景图

页面背景图是显示在每页中的背景图，每页背景图都一样。在设计器中选中【页面视图】，显示页面设置区域，选择【页面背景图】会弹出选择图片对话框



单元格背景图

支持给每个单元格设置一个背景图图片，一个单元格扩展出来的扩展单元格的背景图都一样



单元格图片内容

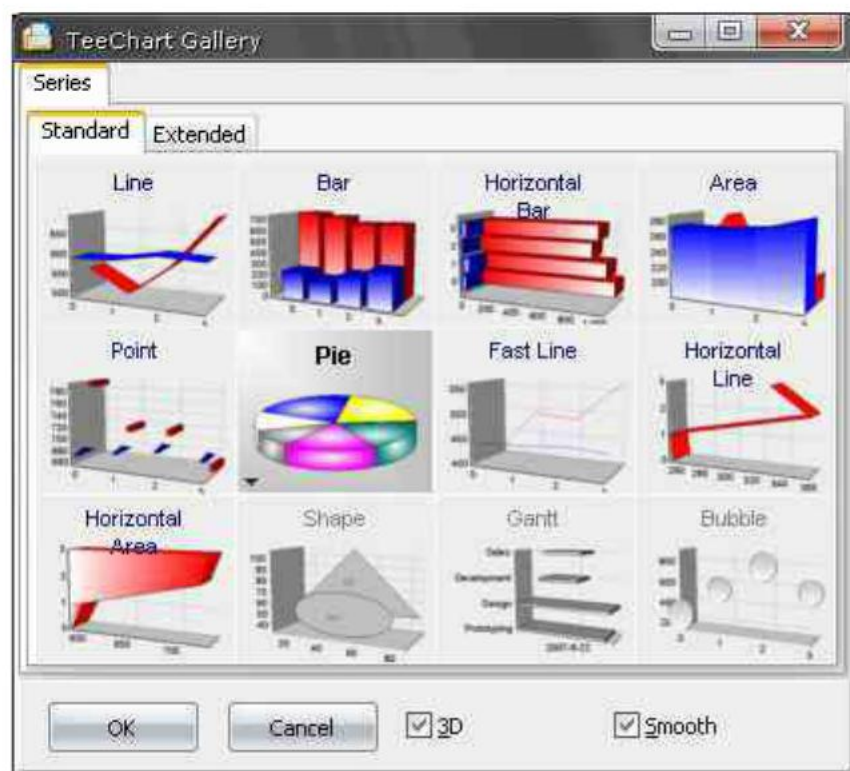
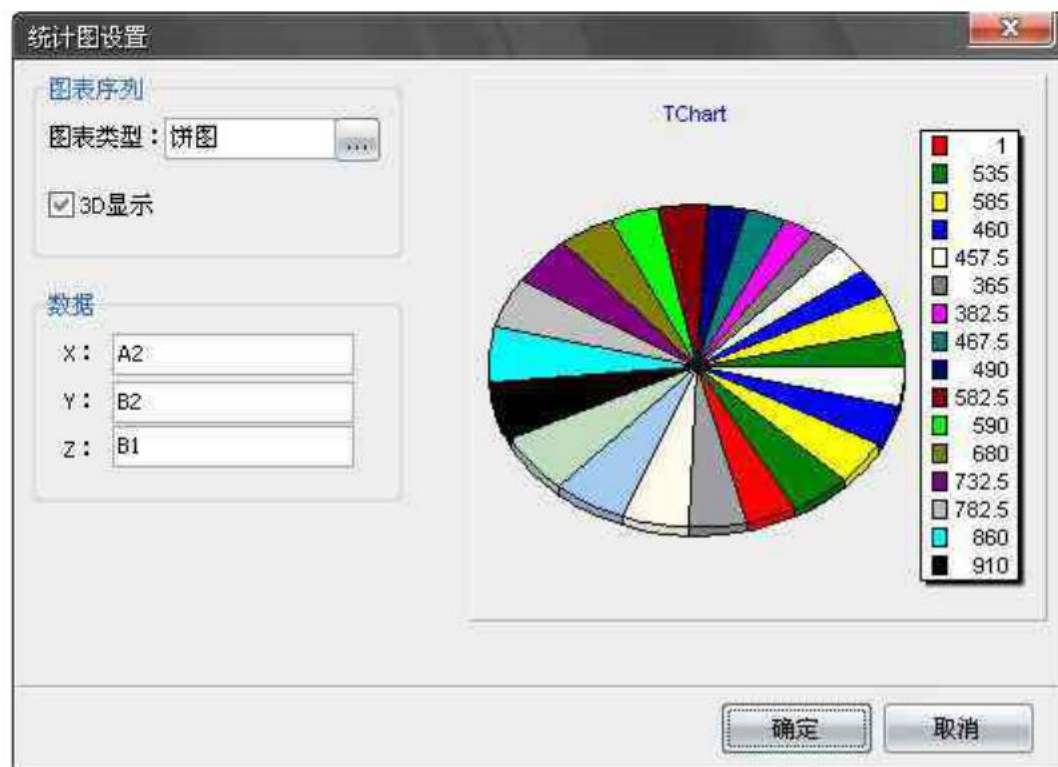
在数据库中会存在图片字段，比如存在一个职工表 employee，每个职工都存入照片 photo，这时可以把设计单元格内容设置为 emplyee.photo，设置单元格数据类型为【图片】，如果报表的字段数据存储遵循报表图形存储格式，则报表引擎会显示成图形。

统计图

由于引擎在 Delphi 下实现，目前报表采用 TeeChart 来作为统计图的生成。

设置

如果把单元格设置成【统计图】数据类型，则在设计器中点击单元格设置按钮不 时，将显示如下设置对话框，在对话框中可以设置图形类型，并设置统计数据的 X、Y 轴和系列 Z 轴。



X、Y、Z 轴表达式由单元格表示，表达式为单元格 Notation，不加表示集合的 ‘[]’

打印模型

介绍

本文档是报表模型的打印相关模型部分

术语

名称	说明
带区	
页眉区	页眉区位于页面的最上面，每页都有，一般用于放置和页面有关的信息，比如：第 X 页/共 X 页、工程名称、公司名称等。
标题区	标题区用于放置报表标题，可以引用数据区单元格的数据。
表眉区	标题和表体之间部分，用来存放表的抬头。分页时每页重复。
表头区	表头区一般用于放置报表数据列的列名称，它和数据区其他单元格不一样的地方在于：分页的时候表头区每页重复，分栏时每栏重复。当报表宽度比纸张显示区域宽时才需要设置表头列，这种情况下分页时表头列就会在每页中重复显示。
数据区	数据区一般放置报表从数据集中取到的数据。
表脚注	表脚注一般用于放置报表的一些汇总统计信息，以及常在报表表尾显示的一些信息，例如：操作员、操作时间、批注等。最后一页数据最后一行显示。
页统计	统计每页的数据，每页都有，单元格可以引用数据区的单元格的数据。分页时每页重复。
表统计	表统计一般用于放置报表当页的一些汇总统计信息，每页都有。表统计的单元格可以引用数据区的单元格的数据。最后一页单元格最后一行显示。
页脚区	页脚区位于页面的最下面，每页都有。一般用于放置和页面有关的信息，比如：页码、作者，日期等。

※分页断开时值重复显示

合并单元格适用本属性，当分页在合并单元格的中间时，是否在两页的该单元格中显示相同的数据

※合并格可分两页显示

合并单元格适用本属性，当分页在合并单元格的中间时，是否允许该单元格断开显示

✳自动换行

当当前单元格的数据值长度超出单元格的宽度时，是否自动换行。

✳纸张尺寸

报表打印或导出时的缺省纸张大小

✳纸张方向

报表打印或导出时的缺省纸张方向

✳纸张高

报表打印或导出时的缺省纸张高

✳纸张宽

报表打印或导出时的缺省纸张宽

✳左边距

报表打印或导出时的缺省左边距

✳右边距

报表打印或导出时的缺省右边距

✳上边距

报表打印或导出时的缺省上边距

✳下边距

报表打印或导出时的缺省下边距

※分栏数

报表打印或导出时的缺省分栏数

※行高列宽单位

单元格行高和列宽的尺寸单位，有像素、毫米、英寸选项

※报表分页方式

报表的分页方式，可以按纸张尺寸分页，也可以按数据行数分页

※每页显示数据行数

如果报表是按照行数分页的，那么可以设定每页显示数据的行数

※打印布局行数

如果报表是按照行数分页的，那么一张打印纸上可能打印多页报表，该参数设定一张打印纸上纵向可以打印几排报表

※打印布局列数

如果报表是按照行数分页的，那么一张打印纸上可能打印多页报表，该参数设定一张打印纸上横向可以打印几排报表

※打印布局顺序

如果报表是按照行数分页的，那么一张打印纸上可能打印多页报表，该参数设定多页报表在打印纸上先横向排还是先纵向排

※页眉页脚打印设置

如果报表是按照行数分页的，那么一张打印纸上可能打印多页报表，该参数设定页眉页脚在每页报表中都显示还是只在一张打印纸上显示一次。

自动折行

- 标题、页统计、表统计不自动折行
- 表头、表脚

扩展

- 标题区的单元格可以引用数据区的单元格的数据，但不参与扩展和复制
- 表头区参与扩展
- 数据区的单元格可以进行扩展和复制
- 表尾区的单元格可以引用数据区的单元格的数据，但不参与扩展和复制

打印规则

编号	名称	数据说明	数据类型	数据规则	数据来源	备注
1	剩余页面用空行填充	设置当前报表分页后剩余页面是否是用空行填充	布尔		用户选择	默认不选中
2	单元格大小自动适应	设置当前报表单元格是否大小自动适应	布尔		用户选择	默认选中
3	允许横向拆页	设置当前报表是否允许横向拆页；横表允许横向拆页，且默认不可用	布尔		用户选择	默认选中
4	允许纵向拆页	设置当前报表是否允许纵向拆页；纵表允许纵向拆页，且默认不可用	布尔		用户选择	默认选中
5	数字自动折行	设置当前报表是否是数字自动折行	布尔		用户选择	默认不选中
6	数值 0 输出为空	设置当前报表数值 0 是否输出为空	布尔		用户选择	默认不选中
7	空字段输出为 "-"	设置当前报表是否是空字段输出为 "-"	布尔		用户选择	默认不选中
8	打印时启用套打	设置当前报表是否套打	布尔		用户选择	默认不选中
9	边框线宽	设置当前报表边			下拉选择	默认线宽为

		框的默认线宽				2
10	边框颜色	设置当前报表边框的默认颜色			下拉选择	默认为黑色
11	使用多栏打印	设置当前报表是否使用多栏打印	布尔		用户选择	默认不选中
12	栏距	设置报表的栏间距	整型	范围 [0,1000]	用户输入	初始化显示为 0
13	页末空隙处理	设置页末空隙处理方式：‘不处理’、‘合并到末尾’、‘平均分配’、‘按比例分配’	文本		下拉选择	默认为‘按比例分配’

交叉表的设计示例

根据上面的架构和模型，实现后交叉表示例如下所示：

数据

设计时	运行时	Orders示例数据	钢筋楼层直径统计	
	FloorName	EntType	Type	Diameter Weight
1	首层	柱	一级钢	8 1
2	首层	柱	一级钢	10 2
3	首层	柱	二级钢	12 3
4	首层	暗柱	一级钢	8 4
5	首层	柱	一级钢	6 8
6	第二层	柱	一级钢	10 9
7	第二层	墙	一级钢	6 5.5
8	第二层	墙	一级钢	6 6.5

报表设计期

设计时	运行时	Orders示例数据	钢筋楼层直径统计		
	A	B	C	D	E
1	楼层名称	构件类型	钢筋总重	Group (Floor.Type[])	钢筋总重
2				Group (Floor.Diameter[])	
3	Group (Floor.FloorName[])	Group (Floor.EntType[])	sum (D3[])	sum (Floor.Weight [Diameter < 10])	sum (D3[])
4		小计	sum (C3[])	sum (D3[])	sum (E3[])
5	全部层汇总	Group (Floor.EntType[])	sum (D5[])	sum (Floor.Weight [])	sum (D5[])
6		合计	sum (C5[])	sum (D5[])	sum (E5[])

报表运行期

设计时	运行时	Orders示例数据	钢筋楼层直径统计					
	A	B	C	D	E	F	G	H
1	楼层名称	构件类型	钢筋总重	一级钢			二级钢	
2				8	6	10	12	钢筋总重
3	首层	柱	12	1	8	0	3	12
4		暗柱	4	4	0	0	0	4
5		小计	16	5	8	0	3	16
6	第二层	柱	0	0	0	0	0	0
7		墙	12	0	12	0	0	12
8		小计	12	0	12	0	0	12
9	全部层汇总	柱	23	1	8	11	3	23
10		墙	12	0	12	0	0	12
11		暗柱	4	4	0	0	0	4
12		合计	39	5	20	11	3	39

附一：[研究润乾报表的实现](#)

使用过很多公司的报表试用版本，个人觉得[润乾报表](#)是一个非常灵活的产品，虽然目前的设计器还只能适用于开发人员：)

在试用润乾报表过程中，认真揣摩她的核心概念，发现自己也可以实现一个类似的报表引擎，所以自己也使用 Delphi 实现了一个显示报表的核心部分的报表引擎（还未添加填报功能），可以看看我以前写的《[交叉表的报表设计](#)》示例

实现报表引擎需要使用用到以下两个个核心技术和概念：

- 数学表达式引擎
- 报表扩展模型

准备重新实现一个 .Net 下的报表引擎，可以基于它之上进行表单设计、查询窗体、报表、处理数据分析

就像以上说的需要基于两个核心技术，所以会先实现一个表达式引擎。我在以前的文章[Antlr 实现表达式引擎的语法文件](#)中写了使用 Antlr 实现的一个示例的主要语法文件。后期我会使用 Antlr 编写一个数学表达式计算引擎，到时会和大家分享。

网上还有其他可以参考的文章和资料，大家可以参考。例如：

- [State of the Art Expression Evaluation](#)
- [An extensible math expression parser with plug-ins](#)
- [Implementing an Excel-like formula engine](#)
- [Flee - Fast Lightweight Expression Evaluator](#)

附二： 开源 — 轻型的表达式引擎 Flee

在《[报表应用思路](#)》中的**主要技术**中 说过表达式引擎对报表来说是很重要的一个技术，在以前的报表系统中表达式引擎也不是我开发的，我是使用我们公司内部开发的一个 delphi 引擎，运行的还 很不错，如果报表转移到 .Net 下我就最好要找一个 .Net 版的表达式引擎了，而且最好更快一些，因为报表计算有时要计算很多次，效率是重要考虑因素之一。

一两年前注意到了这个开源的表达式引擎 — Flee ([Fast Lightweight Expression Evaluator](#))，感兴趣的可以去尝试一下。由于我还是比较早关注的这个，如果有谁知道还有更好的表达式引擎，也请和我分享一下，[OpenExpressApp](#) 中底层引擎中包括报表引擎，所以对表达式引擎的选择也是一件大事：)

优点

1. 速度快，把表达式直接编译成 IL 代码
2. 轻量级，使用简单方便
3. 支持表达式变量解释，和我公司的一样，通过一个方法可以解释自定义变量
4. 文档还行，看后就会用了
5. 开发也有段时间了，相对来说比较稳定

如何使用

由于轻量级又简单，大家可以使用在任何需要表达式解释的地方，可以参考 [Examples](#)

为了方便我把它的主页介绍附在最后面。

Flee 介绍

Project Description

Flee is an expression parser and evaluator for the .NET framework. It allows you to compute the value of string expressions such as $\text{sqrt}(a^2 + b^2)$ at runtime. It uses a custom compiler, strongly-typed expression language, and lightweight codegen to compile expressions directly to IL. This means that expression evaluation is extremely fast and efficient. Try out the [demo](#), which lets you generate images based on expressions, and see for yourself.

Features

- Fast and efficient expression evaluation
- Small, lightweight library
- Compiles expressions to IL using a custom compiler, lightweight codegen, and the `DynamicMethod` class
- Expressions (and the IL generated for them) are garbage-collected when no longer used
- Does not create any dynamic assemblies that stay in memory
- Backed by a comprehensive suite of unit tests
- Culture-sensitive decimal point
- Fine-grained control of what types an expression can use
- Supports all arithmetic operations including the power (^) operator
- Supports string, char, boolean, and floating-point literals
- Supports 32/64 bit, signed/unsigned, and hex integer literals
- Features a true conditional operator
- Supports short-circuited logical operations
- Supports arithmetic, comparison, implicit, and explicit overloaded operators
- Variables of any type can be dynamically defined and used in expressions
- CalculationEngine: Reference other expressions in an expression and recalculate in natural order
- Expressions can index arrays and collections, access fields and properties, and call functions on various types
- Generated IL can be saved to an assembly and viewed with a disassembler

More information

- [Examples](#) to learn how to create and evaluate expressions.
- [Language Reference](#) for information on the expression language.
- [Frequently Asked Questions](#) about using Flee.
- [Article](#) on CodeProject for a more detailed overview.

License

Flee is licensed under the LGPL. This means that as long as you dynamically link (ie: add a reference) to the officially released assemblies, you can use it in commercial and non-commercial applications.

附三：分析能力的 8 个等级

按照不同分析方法所能给人带来的智能程度，把分析能力划分为 8 个等级。

1. 标准报表

回答：发生了什么？什么时候发生的？

示例：月度或季度财务报表

我们都见过报表，它们一般是定期生成，用来回答在某个特定的领域发生了什么。从某种程度上来说它们是有用的，但无法用于制定长期决策。

2. 即席查询

回答：有多少数量？发生了多少次？在哪里？

示例：一周内各天各种门诊的病人数量报告。

即席查询的最大好处是，让你不断提出问题并寻找答案。

3. 多维分析

回答：问题到底出在哪里？我该如何寻找答案？

示例：对各种手机类型的用户进行排序，探查他们的呼叫行为。

通过多维分析(OLAP)的钻取功能，可以让您有初步的发现。钻取功能如同层层剥笋，发现问题所在。

4. 警报

回答：我什么时候该有所反应？现在该做什么？

示例：当销售额落后于目标时，销售总监将收到警报。

警报可以让您知道什么时候出了问题，并当问题再次出现时及时告知您。警报可以通过电子邮件、RSS 订阅、评分卡或仪表盘上的红色信号灯来展示。

5. 统计分析

回答：为什么会出现这种情况？我错失了什么机会？

示例：银行可以弄清楚为什么重新申请房贷的客户在增多。

这时您已经可以进行一些复杂的分析，比如频次分析模型或回归分析等等。统计分析是在历史数据中进行统计并总结规律。

6. 预报

回答：如果持续这种发展趋势，未来会怎么样？还需要多少？什么时候需要？

示例：零售商可以预计特定商品未来一段时间在各个门店的需求量。

预报可以说是最热门的分析应用之一，各行各业都用得到。特别对于供应商来说，能够准确预报需求，就可以让他们合理安排库存，既不会缺货，也不会积压。

7. 预测型建模

回答：接下来会发生什么？它对业务的影响程度如何？

示例：酒店和娱乐行业可以预测哪些 VIP 客户会对特定度假产品有兴趣。

如果您拥有上千万的客户，并希望展开一次市场营销活动，那么哪些人会是最可能响应的客户呢？如何划分出这些客户？哪些客户会流失？预测型建模能够给出解答。

8. 优化

回答：如何把事情做得更好？对于一个复杂问题来说，那种决策是最优的？

示例：在给定了业务上的优先级、资源调配的约束条件以及可用技术的情况下，请您来给出 IT 平台优化的最佳方案，以满足每个用户的需求。

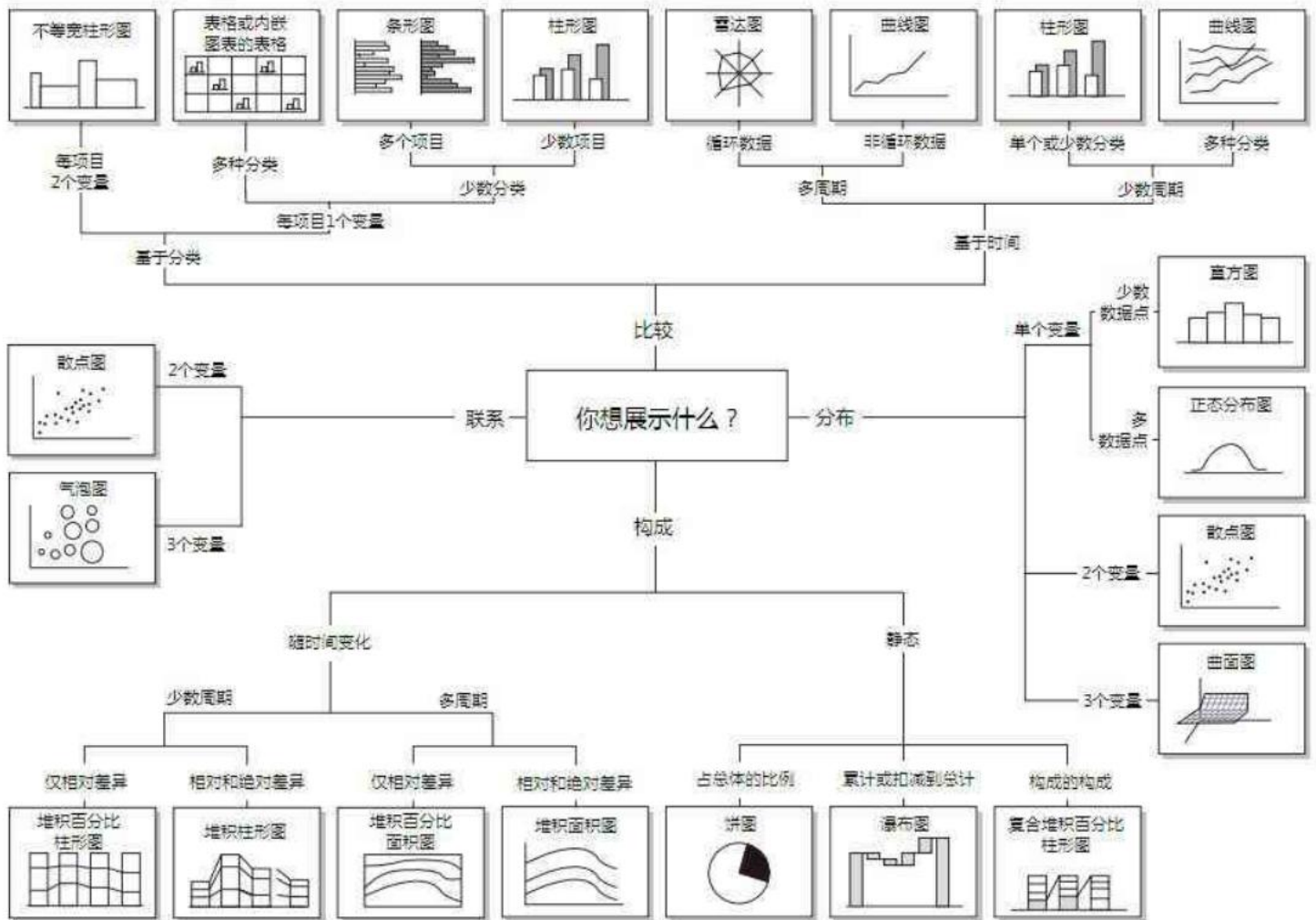
优化带来创新，它同时考虑到资源与需求，帮助您找到实现目标的最佳方式。

本文源自 [Eight levels of analytics](#)，编译：IDMer。

附四：图表建议

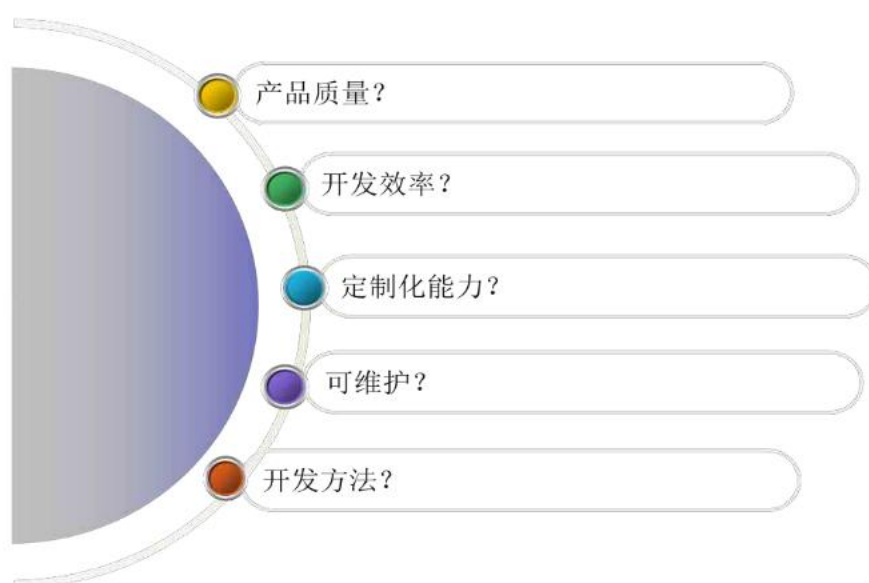
网上看到一个图片，用来指导如何使用图表展现数据

图表建议—思维指南



附五：开源信息系统开发平台一 OpenExpressApp 框架

问题



在多年开发中，对于中大型信息系统的开发总会遇到上面这些问题。从技术角度来说，这些问题的解决好坏往往又决定了该产品是否成功，我认为要想一致的解决这些问题，我们必然要有一个信息系统的开发平台。[开发平台](#)是在系统平台之上，提供在[开发方法](#)指导下，通过 [开发工具](#)、[辅助工具](#)，基于[框架](#)、[引擎](#)以及内置的[模块](#)等一套支持软件[开发生命周期](#)的完整开发环境来构造软件，以期在统一平台下能够快速高质量的提供产品。纯粹的语言开发工具（如 Visual Studio 等）、组件或引擎（如报表引擎等）只是平台的组成部分，单独来说不属于开发平台。开发平台有技术平台，也有业务平台，技术平台的用户更多的是开发人员，而业务平台的用户会着重考虑业务人员，OpenExpressApp 的目标就是做成一个这样的开发平台。

平台如何产生

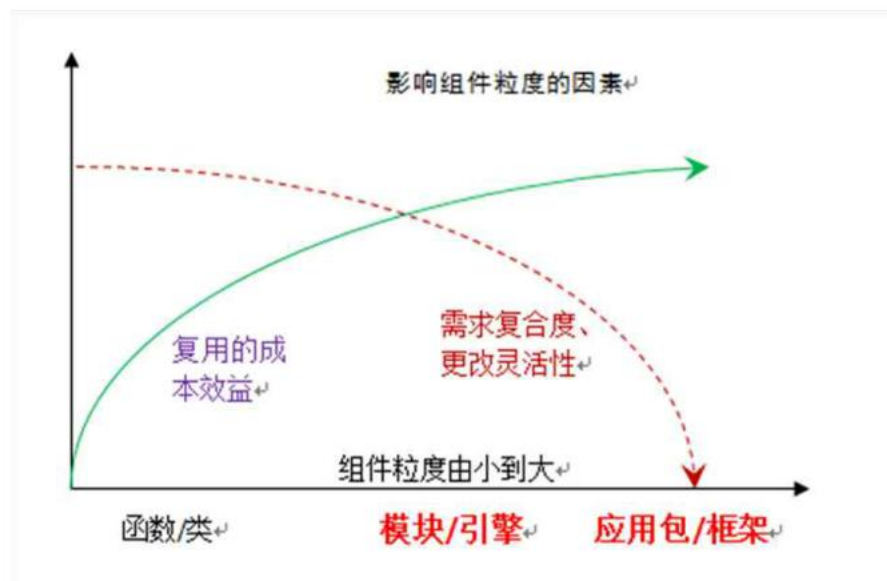
在技术层面，我们经常谈到重用的概念，那么重用又分几个层次呢？我用下图表达了重用的几个层次：

1. 函数和类
2. 模块和引擎
3. 应用包和框架
4. 思想

这几个层次中，在做平台时首要的是思想的重用，框架和引擎的重用都需要思想重用的

指导，而函数和类的重用属于更细节的内容，不属于架构层次。

不同复用层次



思想

基于上面的几个重用层次，平台一般是下面几种方法中产生：

1. 完全自己做
2. 完全用别人的
3. 山寨
4. 扩展
5. 组合 1..4

OpenExpressApp 采用的是第五种方法，通过组合不同引擎、方法、开源产品等来实现一个自己的模型驱动开发平台。

完全自己做



完全用别人的



山寨



扩展

组合



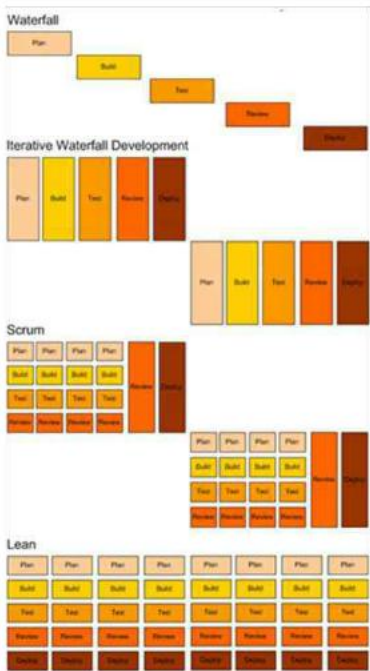
Roadmap

OpenExpressApp 是一个开发平台，提供在[开发方法](#)指导下，通过[开发工具](#)、[辅助工具](#)，基于[框架](#)、[引擎](#)以及内置的[模块](#)等一套支持软件[开发生命周期](#)的完整开发环境。

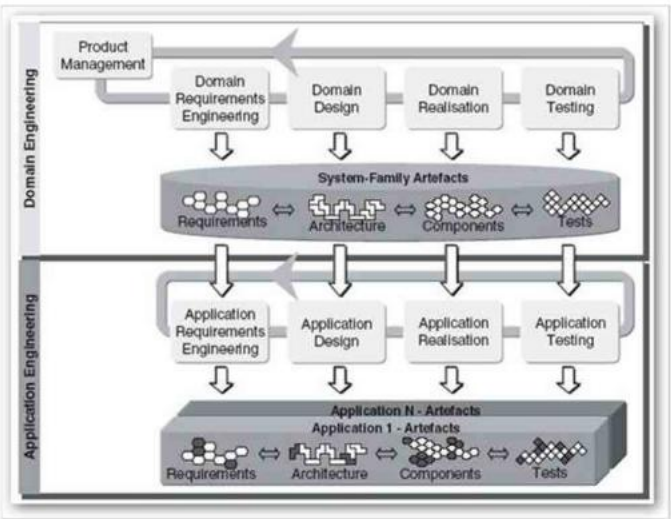
软件产品线工程方法

OpenExpressApp 采用软件产品线工程方法指导，在开发平台时就注意领域开发和应用开发。

流程



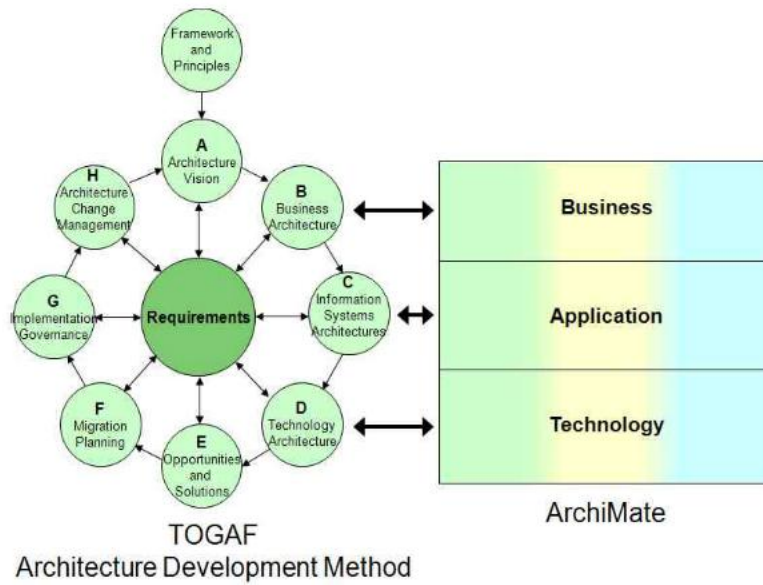
基于单项目开发



基于产品线开发

支持 TOGAF 企业架构开发框架

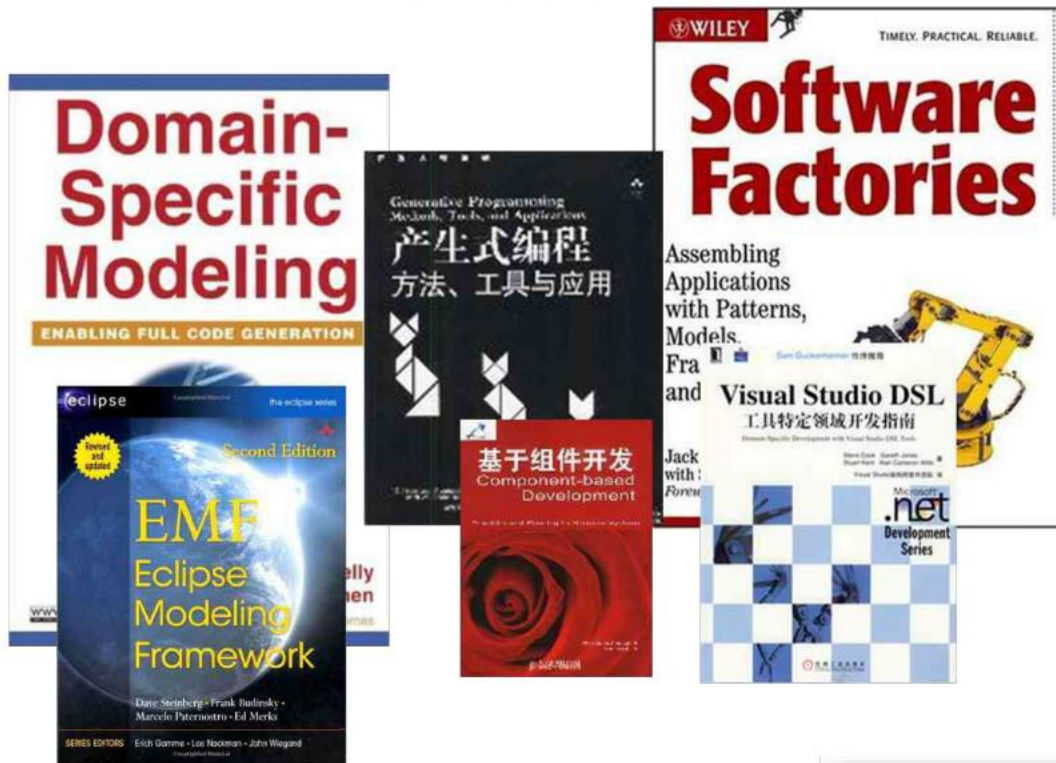
TOGAF 是业界已经公认的企业架构框架，OpenExpressApp 的 EA 建模将支持架构开发框架（ADM），使用 ArchiMate 语言作为建模语言。



从代码为中心到模型驱动

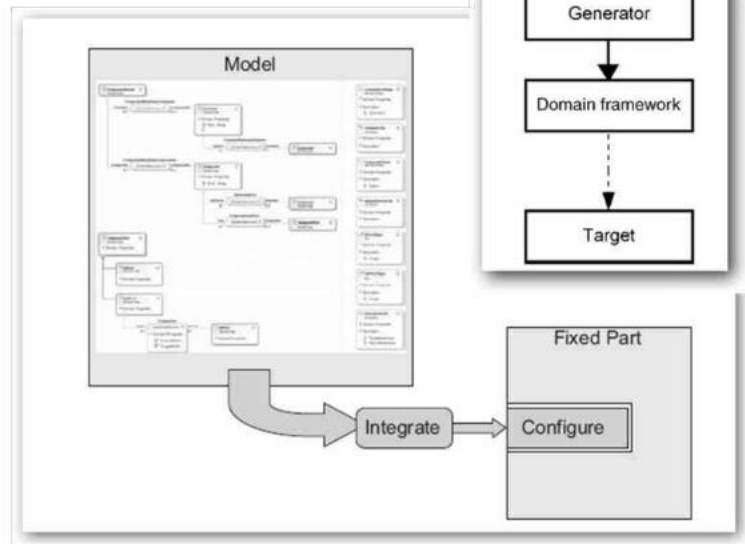
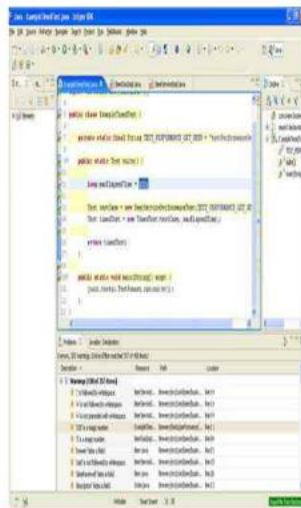
现在业界已经有很多解决规模化问题的开发方法，有软件工厂、产生式编程、DSL、DSM等，其中很多都是基于模型来开发的。

技术方法

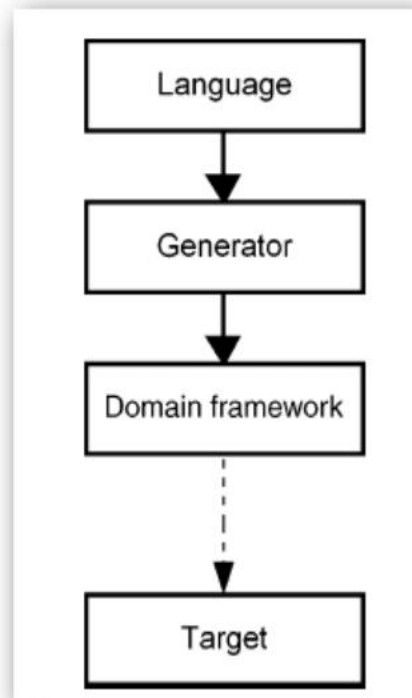


代码

模型



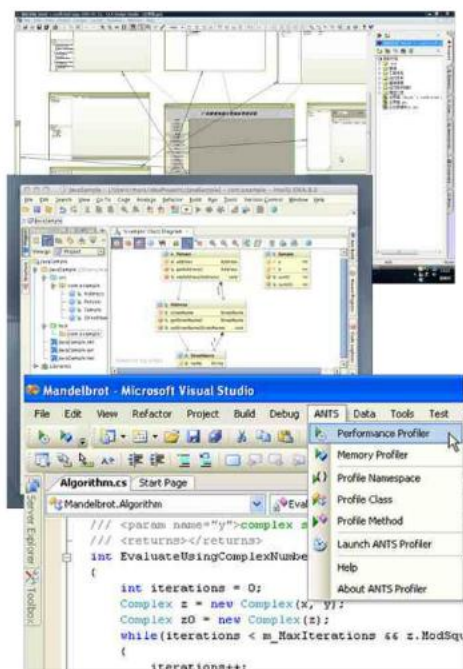
OpenExpressApp 将借鉴上面这些开发方法，主要以 DSM 方法作为指导：



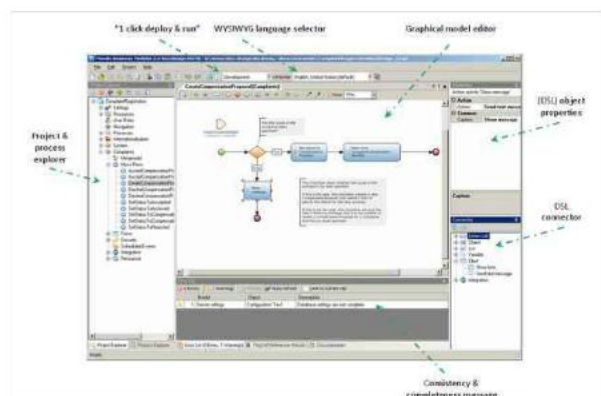
业务集成开发 Studio

以往不同阶段工作借助与不同工具，这样很难让大家协同起来，OpneExpressApp 将提供一个以业务集成平台为中心的开发 Studio，这将很好的提高产品的一致性。

以独立技术为中心



以业务集成平台为中心



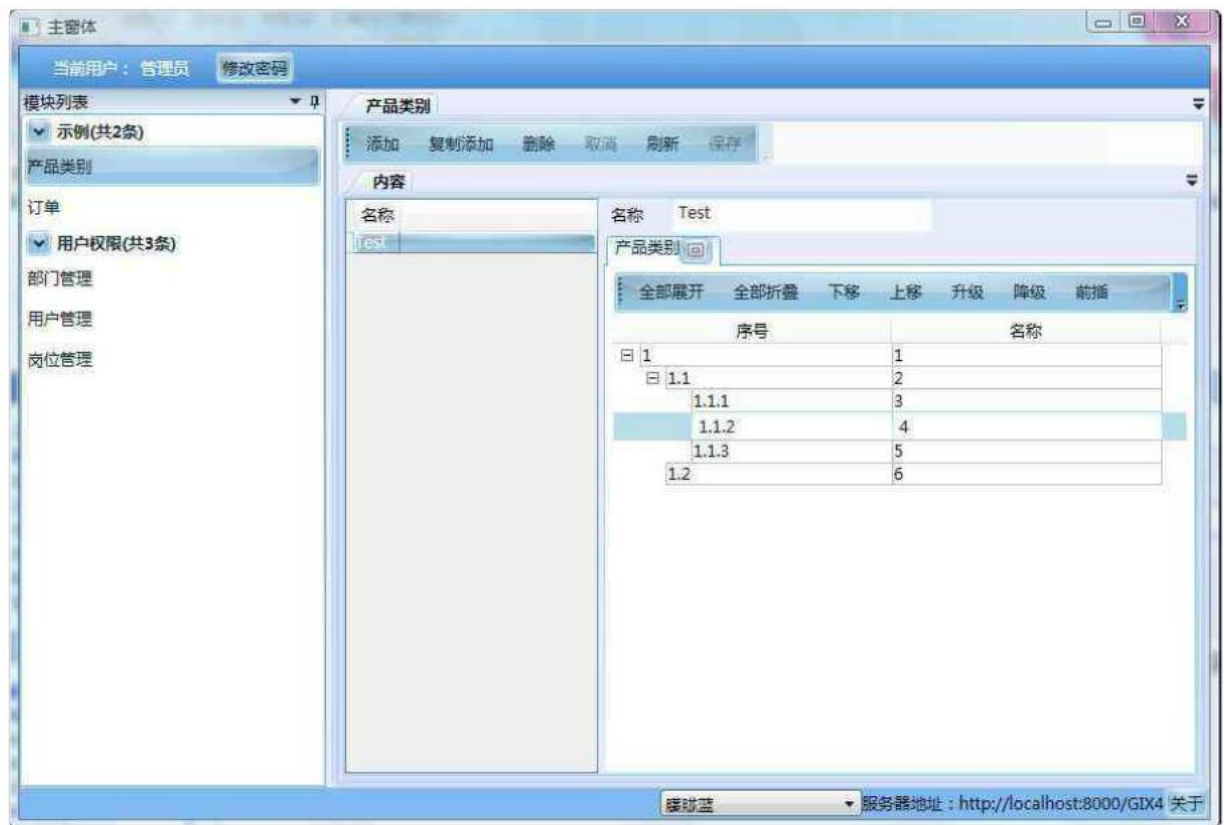
通过以上一些说明可以看出，OpenExpressApp 是将现有技术和产品中有价值的部分引入到一起，它要做的是整合别人已经实践的方法来提高我们自身的开发能力。不同于以往为特定开发角色提供独立的开发工具和框架，它基于业务模型驱动开发指导思想，为业务分析人员、软件设计和开发人员提供的一种集成的开发平台，提供报表、流程、元数据等基础引擎，具有业务建模、领域建模和应用建模等模型，并提供权限、报表等多个通用应用模块。为了支持软件开发，还提供基于敏捷思想、产品线工程的软件项目管理 工具、快速原型工具和测试支持。



- OpenExpressApp 框架(参考 XAF)
- OpenMetaEdit (参考 MetaEdit+)
- OpenReport (参考润乾报表)
- OpenWorkflow (使用微软的 WF)
- OpenStudio (基于 OpenExpressApp 开发, 参考 Mendix、Novulo)
- OpenTool (基于 OpenExpressApp 开发)

当前进展

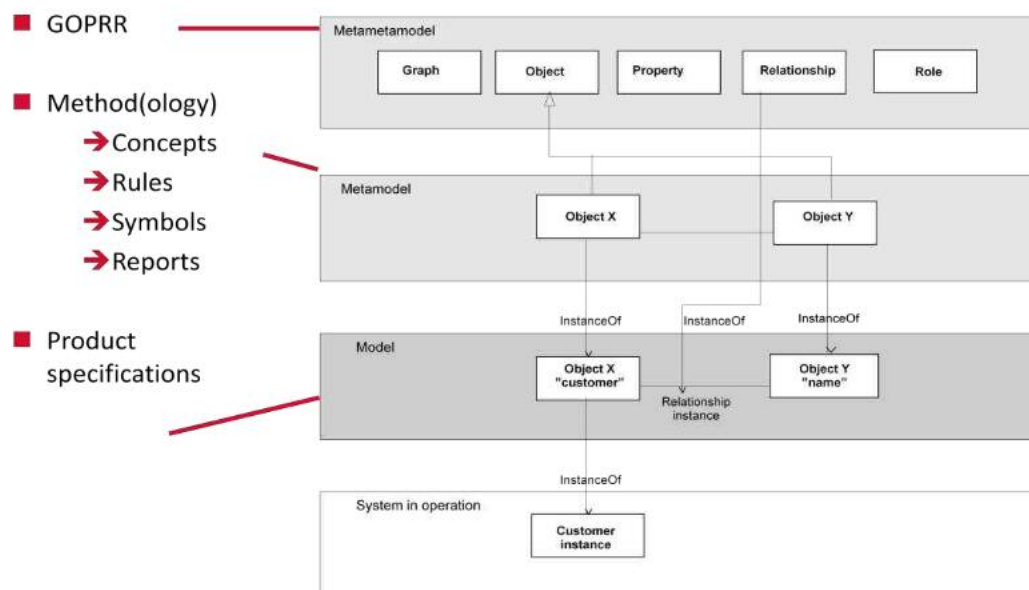
目前处于 OpenExpressApp 框架阶段，提供类似 ExpressAppFramework 的开发框架，供软件开发人员使用，现有框架已在实际项目中应用。现阶段实现主要是基于 C/S(WPF)分布式，参考了 ExpressApp Framework (应用框架)，使用了 CSLA 作为分布式业务对象框架，Composite WPF 作为 UI 框架，还使用了其他一些代码 (如 LiteORM、Flee 等)。



下一步工作

元模型引擎 OpenMetaEdit

Model architecture in MetaEdit+



Metamodeling concepts of GOPRR

■ Concepts

➔ Graph

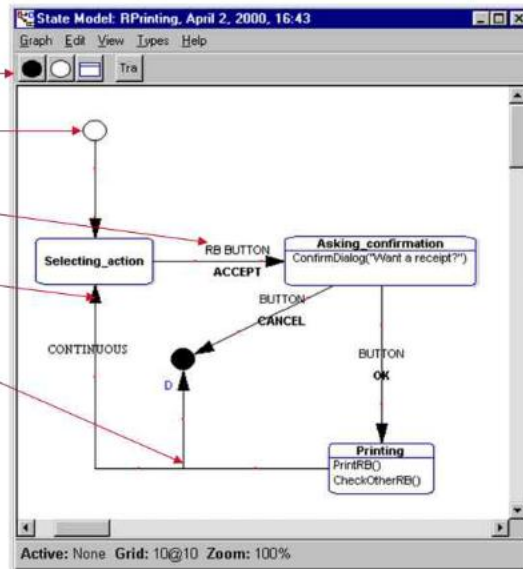
➔ Object

➔ Property

➔ Role

➔ Relationship

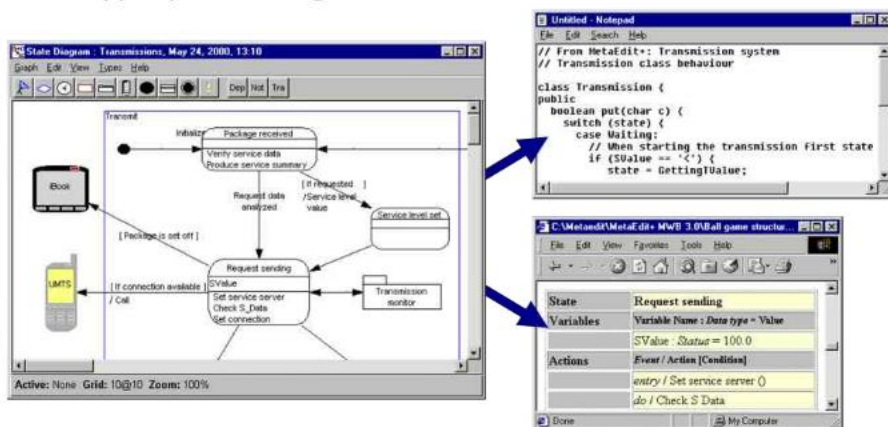
■ MetaEdit+ has own tool for each metamodeling concept



41

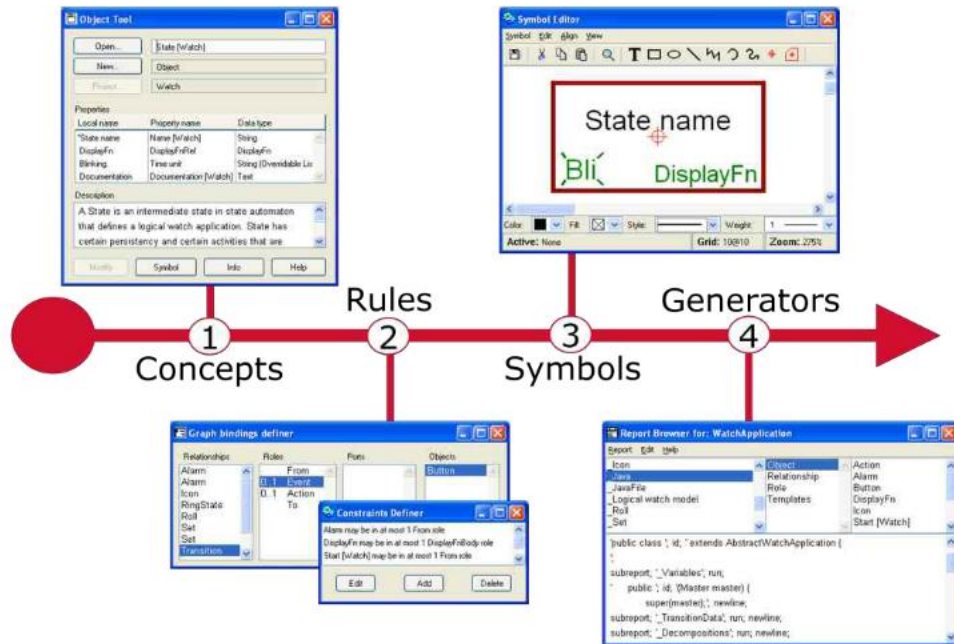
Apply in software production

- MetaEdit+ delivers immediately the method to your team (editors, reports, multi-user, multi-platform, etc)
- Prototype your design; refine and iterate



42

Steps for implementing DSM



43

代码下载

源码页面:

<http://openexpressapp.codeplex.com/SourceControl/list/changesets>

SVN 地址: <https://OpenExpressApp.svn.codeplex.com/svn>

项目站点: [Codeplex](http://codeplex.com)

春节飞虎版 1.1 发布使用说明

1. 生成数据库: 执行 Example\OpenExpressApp.sql 生成数据库, 或者通过 Lib\DBCcreator.exe 从项目编译后的类库生成数据库
2. 打开 OpenExpressApp\OpenExpressApp.sln, 修改 OpenExpressApp.Host.WPF 配置文件的数据 库连接字符串为自己的配置
3. 运行